

# Data visualisation, mean separation & Continuous data

## TRR259 seminar session 2

JS

12.11.2025

# Outline for the second session

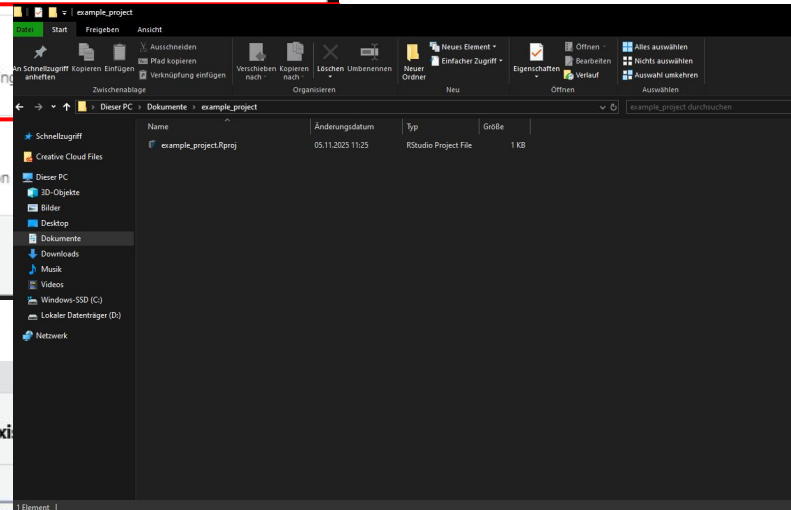
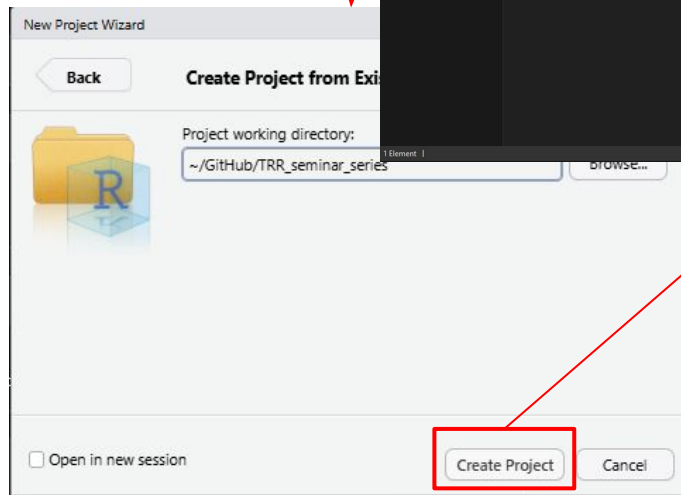
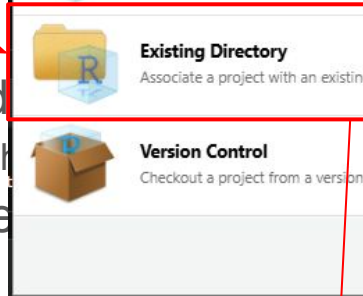
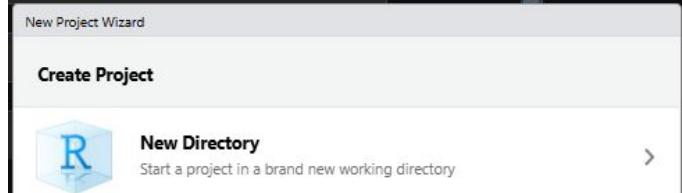
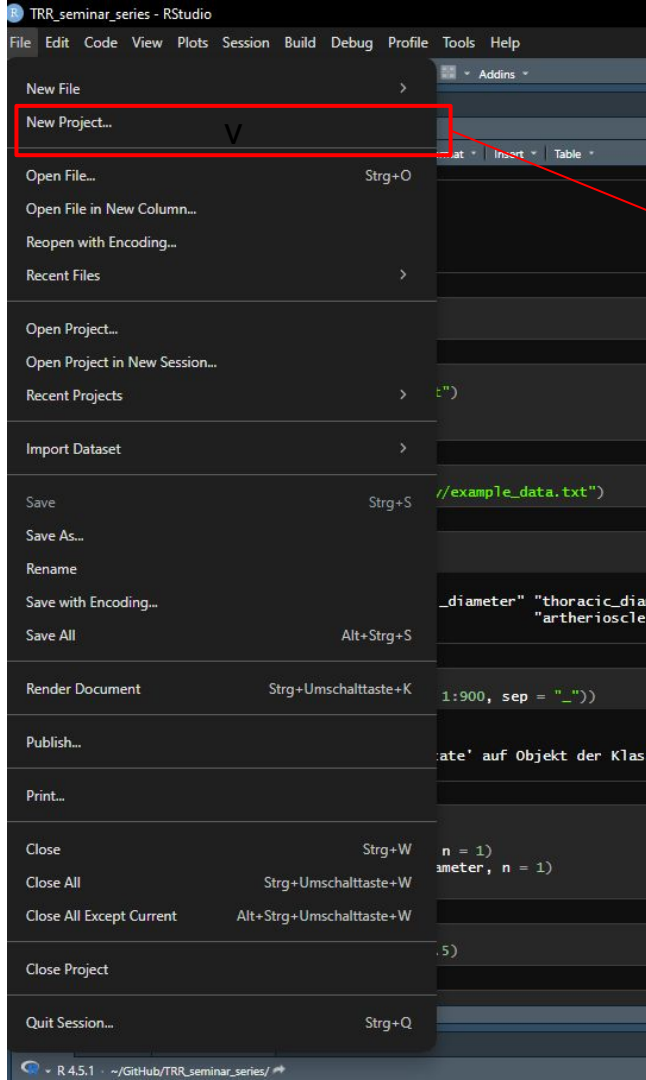
1. R projects, Notebooks and Research data organisation
2. ggplot as a grammar of visualisation
3. Mean Separation and statistics

# Data analysis using R (Studio)

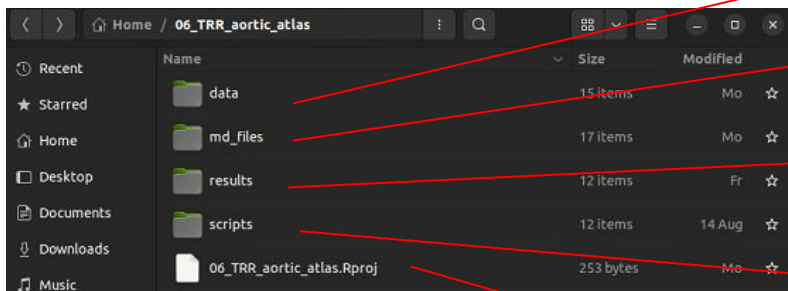
Computational research comes with its own organisational troubles:

- scripts for analysis, oftentimes multiple versions
- input data that everything is based on
- results, both visual and not visual
- intermediate products such as processed data, ...

→ Need for organisation: The Project to the rescue!



# Inside a project folder



all the input to our analysis

the notebooks creating the analysis

all the results

optional: scripts for self-contained code

the project file (click to open)

**Structure is not self-maintaining. You need to maintain it!**

# Guidelines for Projects

- every project gets its own folder
- every analysis gets its own notebook
  - one for RNA seq, one for FACS from the same project
- name your documents / files a sensible way.

Goal: come back in 2 years time, still know where your files are and how you generated results.

Go make a project for this course!

# the here package

- very simple [package](#) designed for work inside projects
- `here()` returns the path to your project

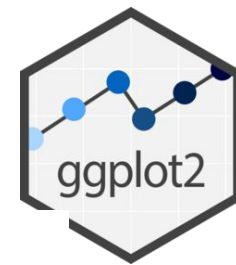
```
{r}  
here()  
  
[1] "C:/Users/Jasper/Documents/GitHub/TRR_seminar_series"
```

- to locate a file in your project, put the path relative to the project directory
- can be used inside functions like `read_tsv()`
- the project can be moved without issue

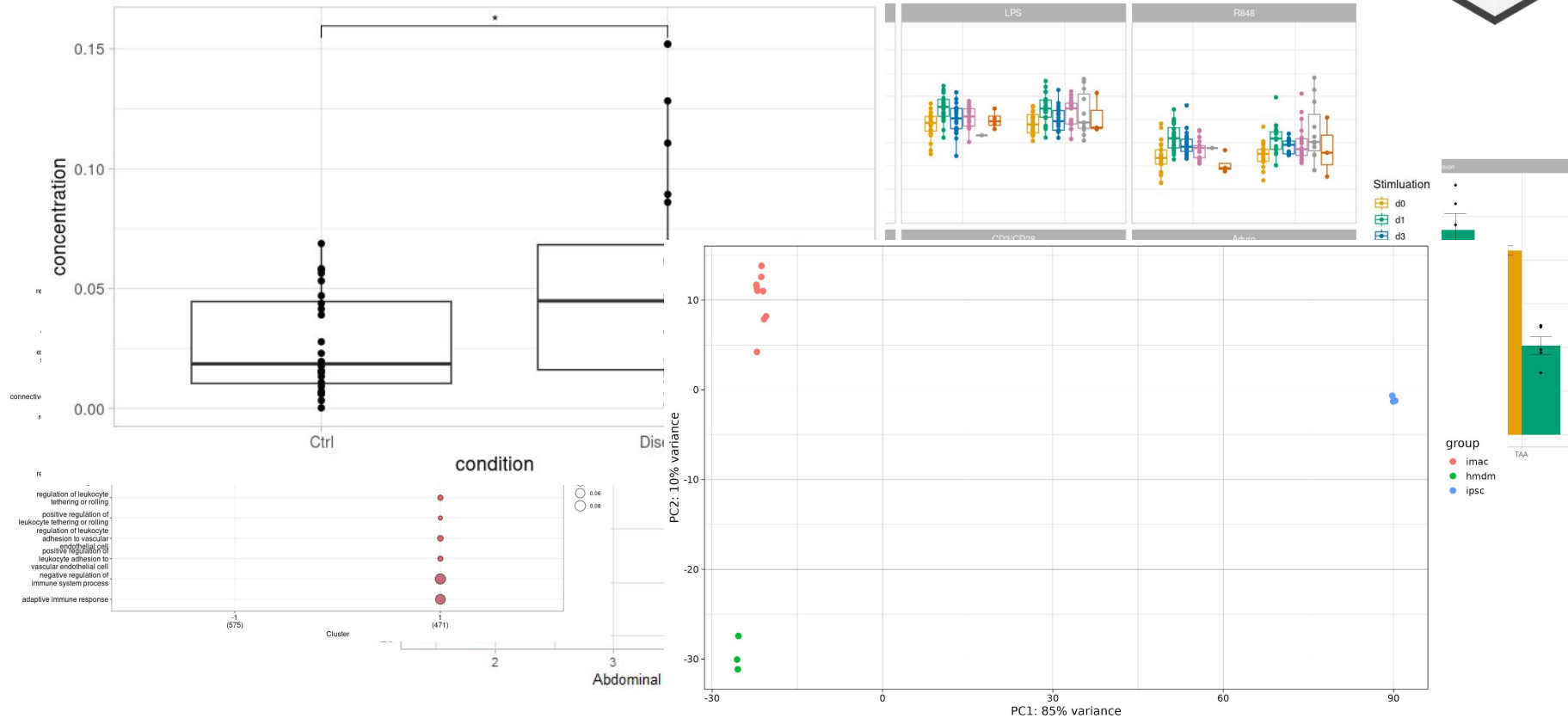
```
{r}  
read_tsv(here("data/aneurysm_survey/example_data.txt"))  
  
Rows: 900 Columns: 9  
— Column specification  
  
Delimiter: "\t"  
chr (1): diagnosis  
dbl (8): abdominal_diameter, thoracic_diameter, age, male, hypertension, cad,  
arteriosclerosis, bmi  
  
i Use 'spec()' to retrieve the full column specification for this data.  
i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```



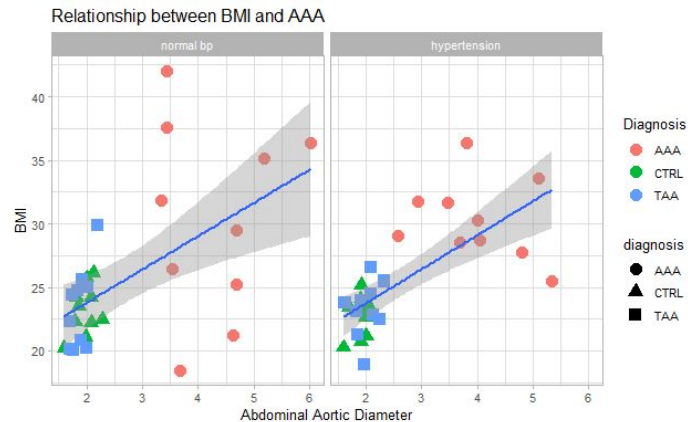
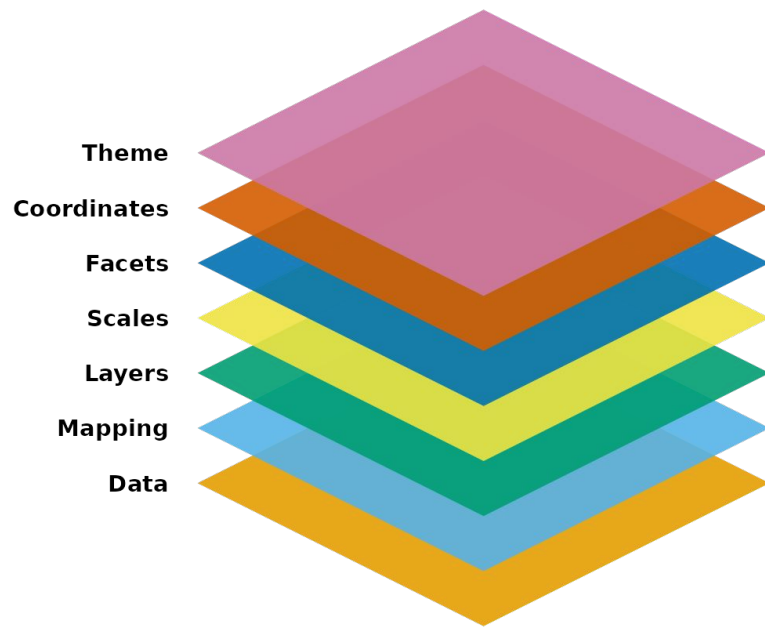
# ggplot2 - Data visualisation

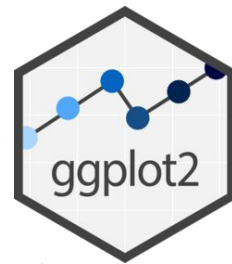


IFN $\gamma$  concentration on d0, unstim



# ggplot2 - A grammar for graphics





# ggplot2 - A grammar for graphics - data

*ggplot()* needs data to create visualisations, so we pass some using the *ggplot(data = XXX)* format.

```
df <- read_tsv(here("data",  
                    "aneurysm_survey",  
                    "example_data.txt"))  
  
df_sub <- df %>%  
  slice_sample(n = 10,  
              by = c(diagnosis, hypertension))  
  
ggplot(data = df_sub)  
  
df_sub %>% ggplot()
```

Reading in our data

Subsetting the data for the visual

creating a new *ggplot*, with *df\_sub* as the data

another way to pass data to *ggplot*



# ggplot2 - A grammar for graphics - data

*ggplot()* needs data to create visualisations so we pass some using the *ggplot(data = XXX)* format.

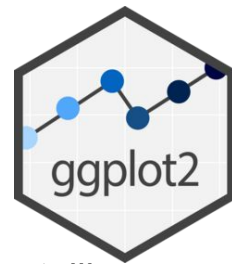
```
df <- read_csv("data.csv")
```

```
df_sub <- df %>%  
  filter(x > 10)  
  by
```

```
ggplot(data = df_sub)
```

```
df_sub %>% ggplot(aes(x = x, y = y))
```

the data



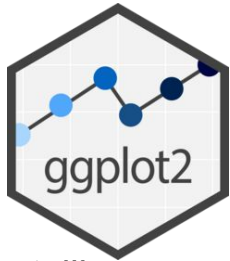
# ggplot2 - A grammar for graphics - data

In *ggplot()*, mapping is the act of taking data and translating it into different visualisations. You are telling the system, what variable maps to what in your plot. For that, we are using the aesthetics, *aes()*, function.

```
df_sub %>%  
  ggplot(aes(x = abdominal_diameter,  
             y = bmi))  
  
df_sub %>%  
  ggplot(aes(abdominal_diameter, bmi))
```

Passing the data  
mapping the data, abdominal diameter to X, bmi to Y

Same thing, but shorter. X is assumed to be first, y is assumed to be second



# ggplot2 - A grammar for graphics - data

In *ggplot()*, mapping is the act of taking data and translating it into different visualizations. You are telling the system, w

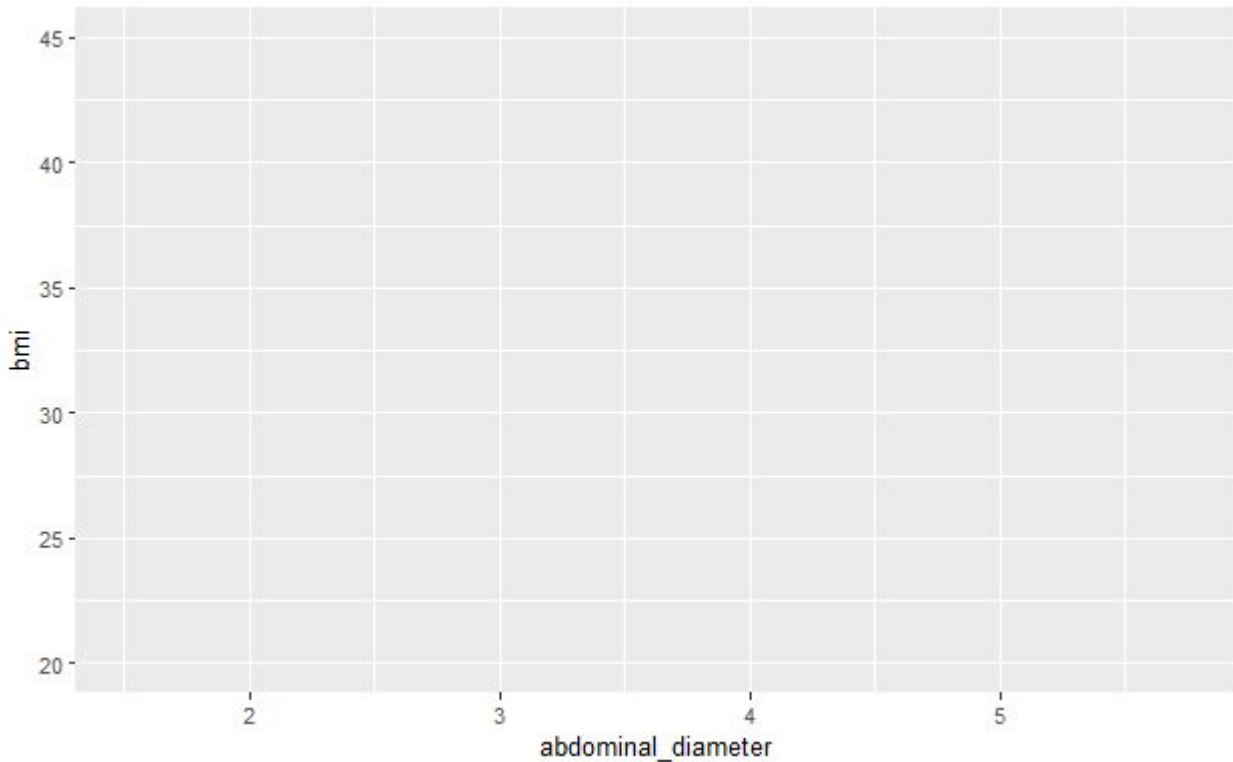
*aes()*, function.

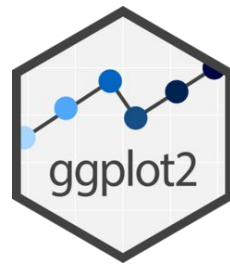
```
df_sub %>%  
  ggplot()
```

r to X, bmi to Y

```
df_sub %>%  
  ggplot()
```

to be first, y is





# ggplot2 - A grammar for graphics - layers

The layers in ggplot determines how the mapped data is being displayed. It has three parts:

1. Geometry or geom - how is the data displayed.
2. Statistical transformation - what is displayed.
3. Positional adjustment - where it is displayed.

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi))+  
  geom_point()
```

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi))+  
  geom_point()+  
  geom_smooth(method = "lm")
```

Passing the data

mapping the data, abdominal diameter to X, bmi to Y  
creating a scatterplot

Passing the data

mapping the data, abdominal diameter to X, bmi to Y  
creating a scatterplot  
adding a regression line

# The layers of a ggplot

The different ways to visualise data can make use of different aesthetics

For most geoms (geometries), there is an X and a Y aesthetic.

*geom\_point()* has the following aesthetics:

- **x** - x position
- **y** - y position
- **alpha** - how transparent is the point
- **color** - the color of the point outline
- **fill** - the color of the filling of the point
- **shape** - shape of the point
- **size** - size of the point



# Layers and aesthetics

Aesthetics can be mapped using the *aes()* function, or set to a manual value

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi,  
    color = diagnosis))+  
  geom_point()
```

Passing the data  
mapping the data, abdominal diameter to X, bmi to Y  
and color to diagnosis  
creating a scatterplot

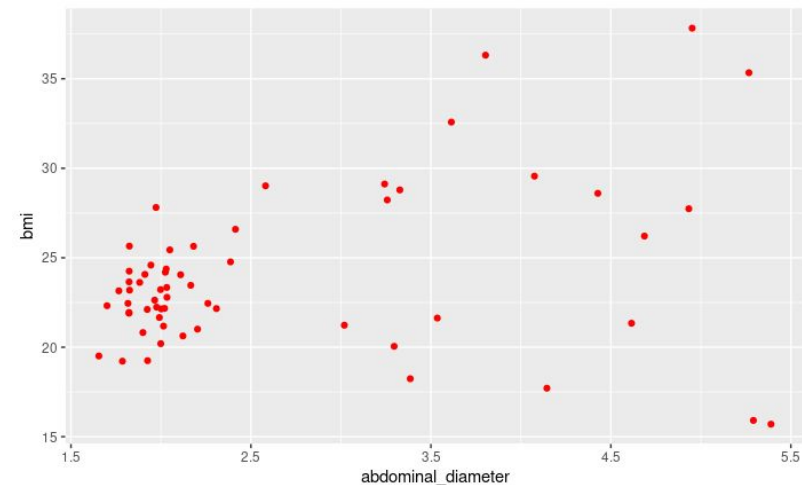
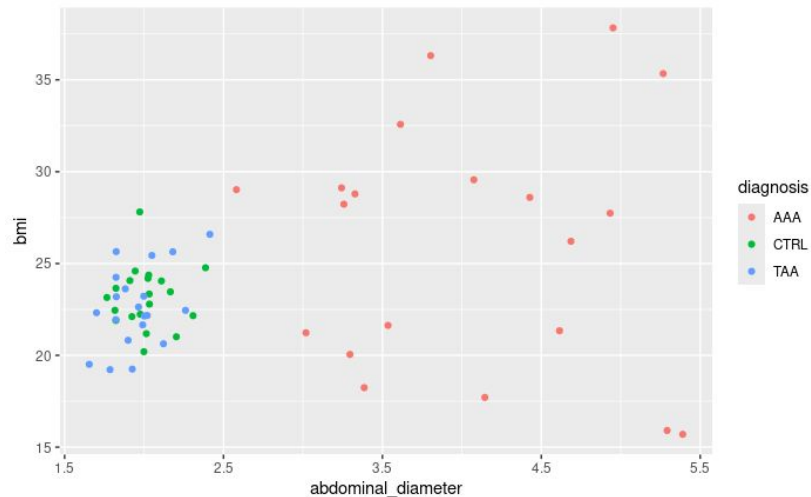
```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi))+  
  geom_point(color = "red")
```

Passing the data  
mapping the data, abdominal diameter to X, bmi to Y  
creating a scatterplot, setting the color to red

# Layers and aesthetics

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi,  
    color = diagnosis))+  
  geom_point()
```

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi))+  
  geom_point(color = "red")
```





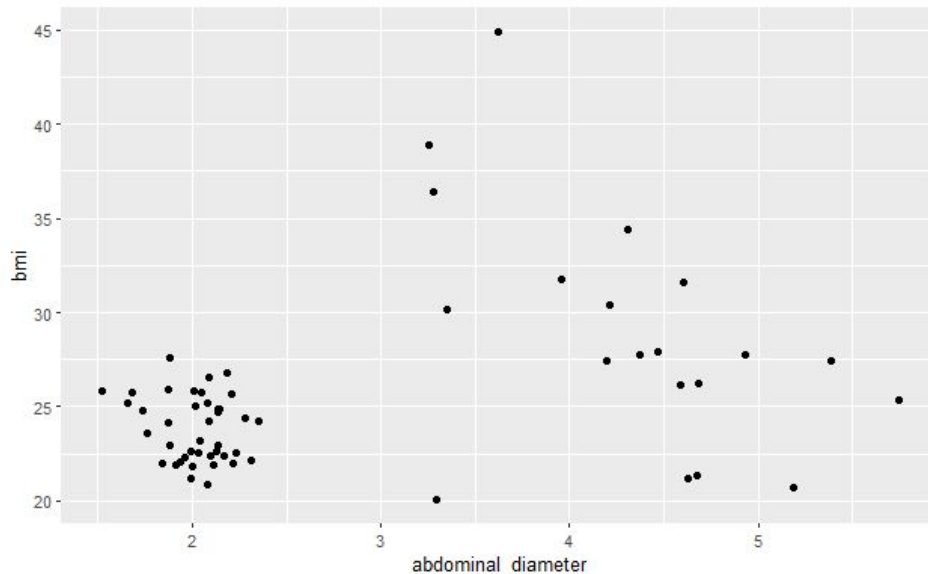
# ggplot2 - A grammar for graphics - layers

The layers in ggplot determines how the mapped data is being displayed. It has three parts:

1. Geometry or geom - how is the data displayed.
2. Statistical transformation - what is displayed.
3. Positional adjustment - where it is displayed.

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi)) +  
  geom_point()
```

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi)) +  
  geom_point() +  
  geom_smooth(method = "lm")
```





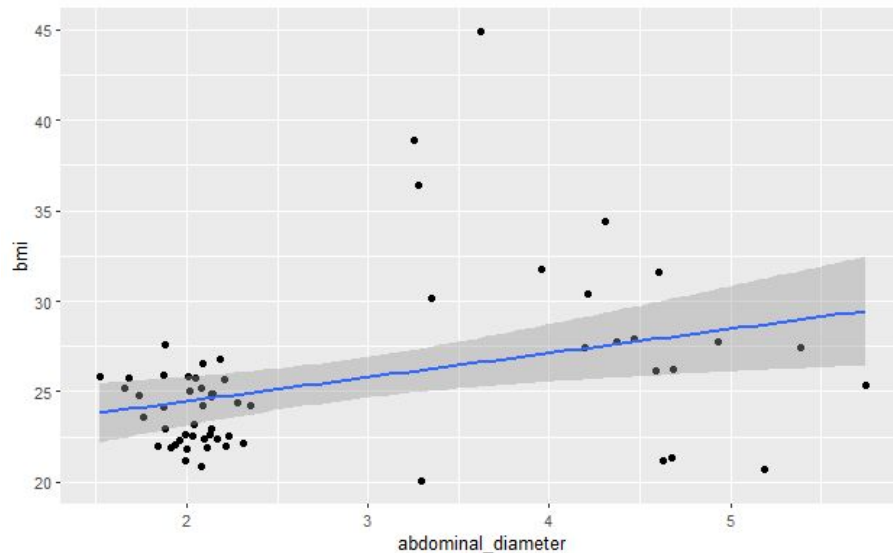
# ggplot2 - A grammar for graphics - layers

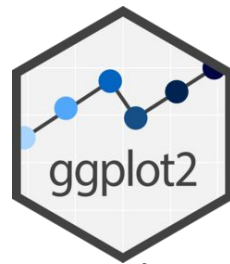
The layers in ggplot determines how the mapped data is being displayed. It has three parts:

1. Geometry or geom - how is the data displayed.
2. Statistical transformation - what is displayed.
3. Positional adjustment - where it is displayed.

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi))+  
  geom_point()
```

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi))+  
  geom_point()+  
  geom_smooth(method = "lm")
```





# ggplot2 - A grammar for graphics - scales

The scales in ggplot are used to transform visuals back to understanding. Everything to do with axes and legends is down to scales.

The general format is: `scale_{aesthetic}_{type}()` where the aesthetic is decided by the `aes()` function of the plot.

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi,  
      color= diagnosis))+  
  geom_point()
```

Passing the data  
mapping the data, abdominal diameter to X, bmi to Y  
and color to diagnosis  
creating a scatterplot

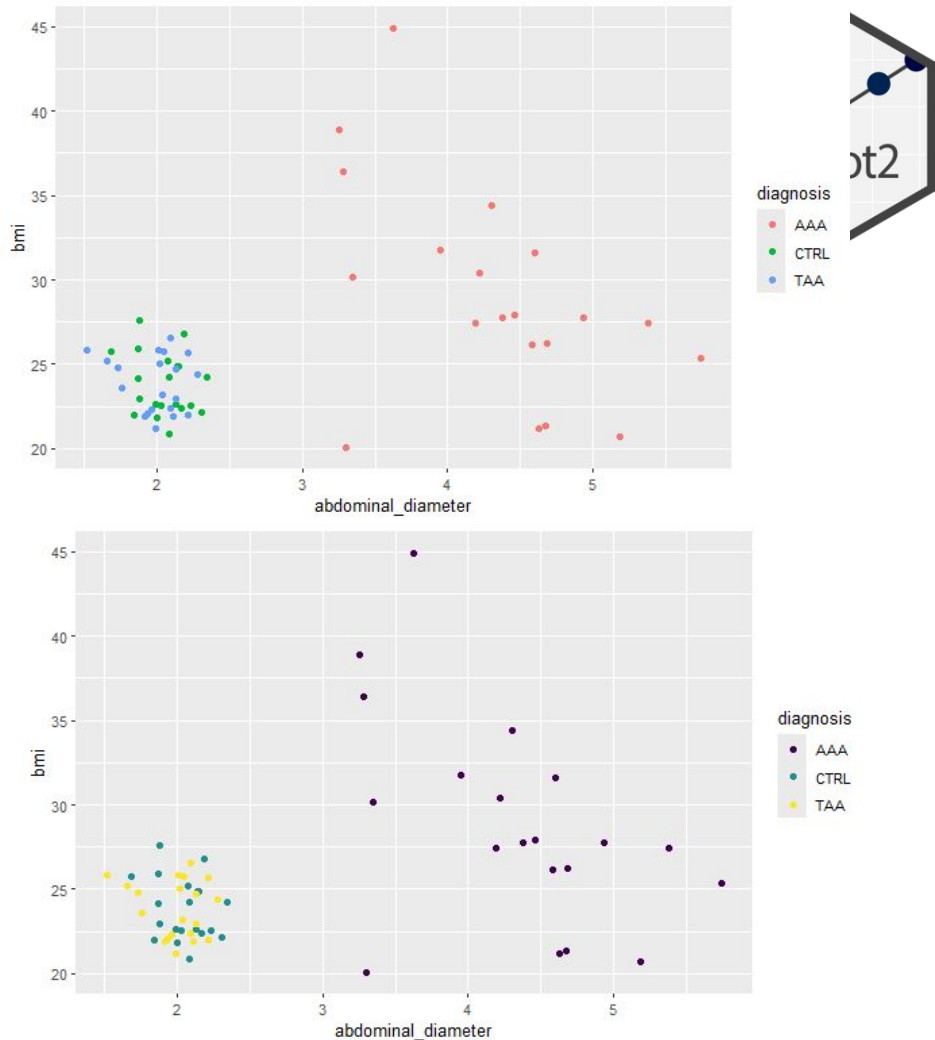
```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi,  
      color= diagnosis))+  
  geom_point()+  
  scale_color_viridis_d()
```

Passing the data  
mapping the data, abdominal diameter to X, bmi to Y and  
color to diagnosis  
creating a scatterplot  
changing the color scale

# ggplot2 - A grammar for graphics

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi,  
      color= diagnosis))+  
  geom_point()
```

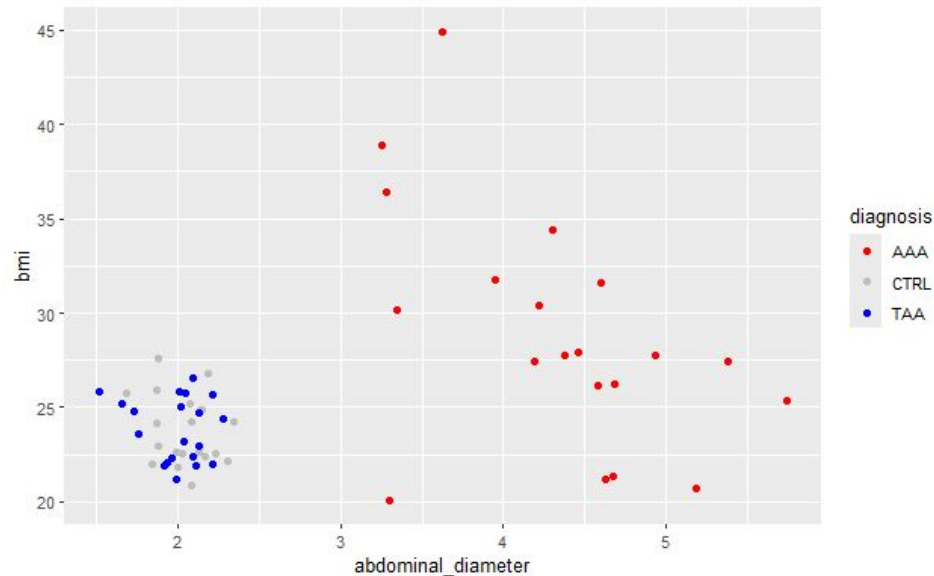
```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi,  
      color= diagnosis))+  
  geom_point()+  
  scale_color_viridis_d()
```





# ggplot2 - A grammar for graphics - scales

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi,  
    color= diagnosis))+  
geom_point()+  
scale_color_manual(  
  values = c("red", "grey", "blue"))
```





# ggplot2 - A grammar for graphics - scales

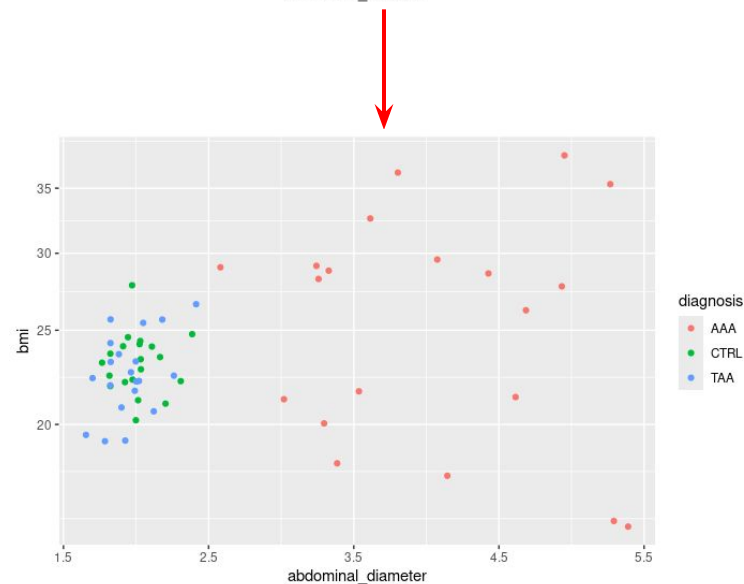
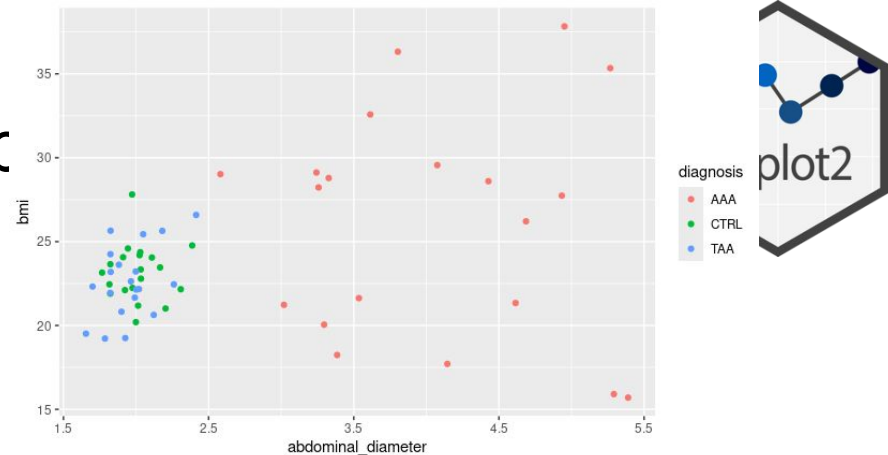
```
df_sub %>%  
  ggplot(aes(abdominal_diameter,  
             bmi, color = diagnosis))+  
  geom_point()+  
  scale_y_log10()
```

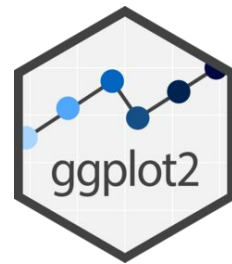
Passing the data  
mapping the data, abdominal diameter to X, bmi to Y  
and color to diagnosis  
creating a scatterplot  
changing the y axis to log10 scale



# ggplot2 - A grammar for graphs

```
df_sub %>%  
  ggplot(aes(abdominal_diameter,  
             bmi, color = diagnosis))+  
  geom_point()+  
  scale_y_log10()
```





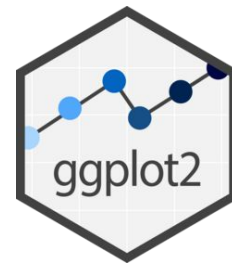
# ggplot2 - A grammar for graphics - facets

Facets are a way to split your data and your plots into multiple, smaller subplots.

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi,  
      color= diagnosis))+  
geom_point()+  
scale_color_manual(  
  values = c("red", "grey",  
            "blue"))+  
facet_wrap(~hypertension)
```

Passing the data  
mapping the data, abdominal diameter to X, bmi to Y  
and color to diagnosis  
creating a scatterplot  
adding our custom color code

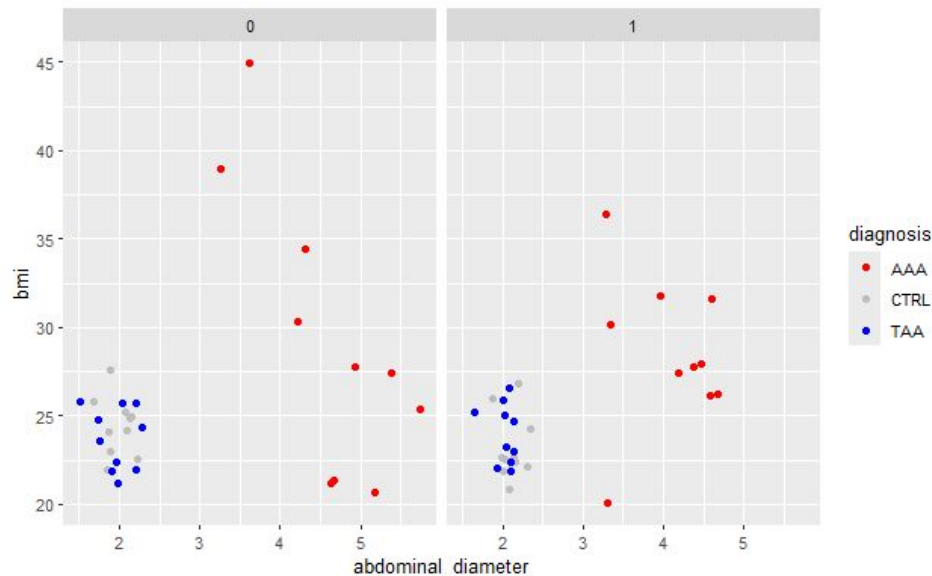
applying the faceting

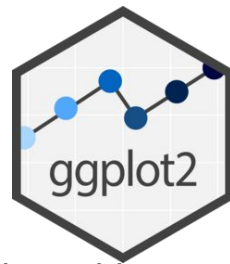


# ggplot2 - A grammar for graphics - facets

Facets are a way to split your data and your plots into multiple, smaller subplots.

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi,  
    color= diagnosis))+  
  geom_point()+  
  scale_color_manual(  
    values = c("red", "grey",  
      "blue"))+  
  facet_wrap(~hypertension)
```





# ggplot2 - A grammar for graphics - themes

Theming in ggplot is about the looks of anything that is independent of the data - background color, grid lines, font sizes, legend positions ...

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi,  
      color= diagnosis))+  
geom_point()+  
scale_color_manual(  
  values = c("red", "grey", "blue"))+  
theme(  
  legend.position = "bottom",  
  axis.line = element_line(linewidth = 0.75),  
  axis.line.x =  
    element_line(color = "blue")  
  axis.text.x = element_text(angle = 45,  
                              hjust = 1)
```

Passing the data  
mapping the data, abdominal diameter to X,  
bmi to Y and color to diagnosis  
creating a scatterplot  
adding our custom color code

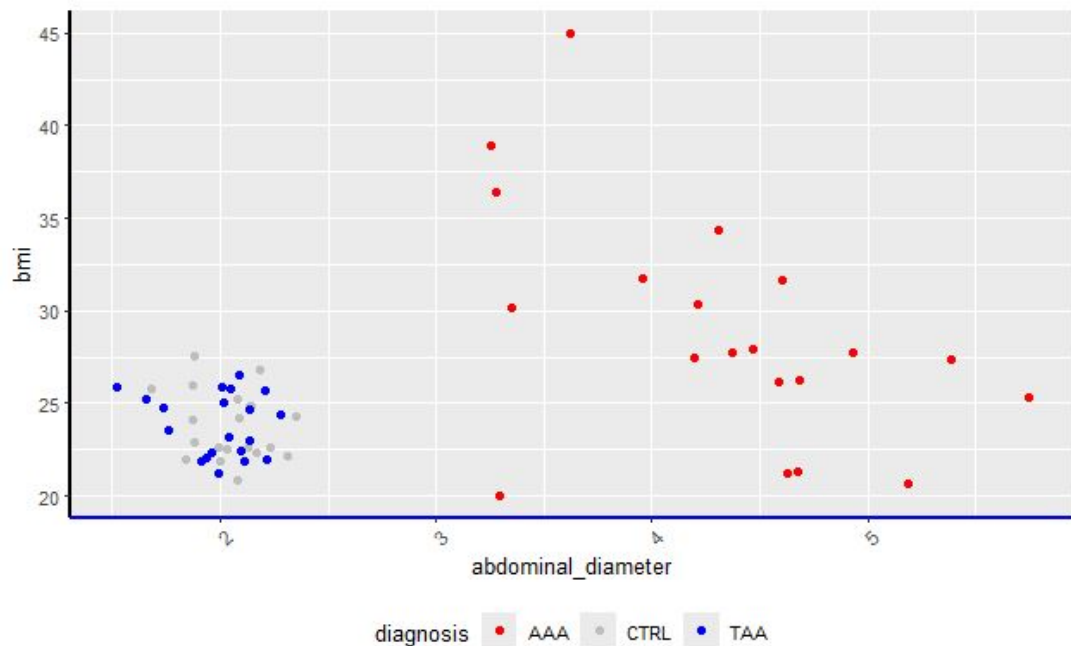
accessing theming  
adjusting legend position  
adjusting line width  
adjusting line color

adjusting the axis labels in angle and  
position

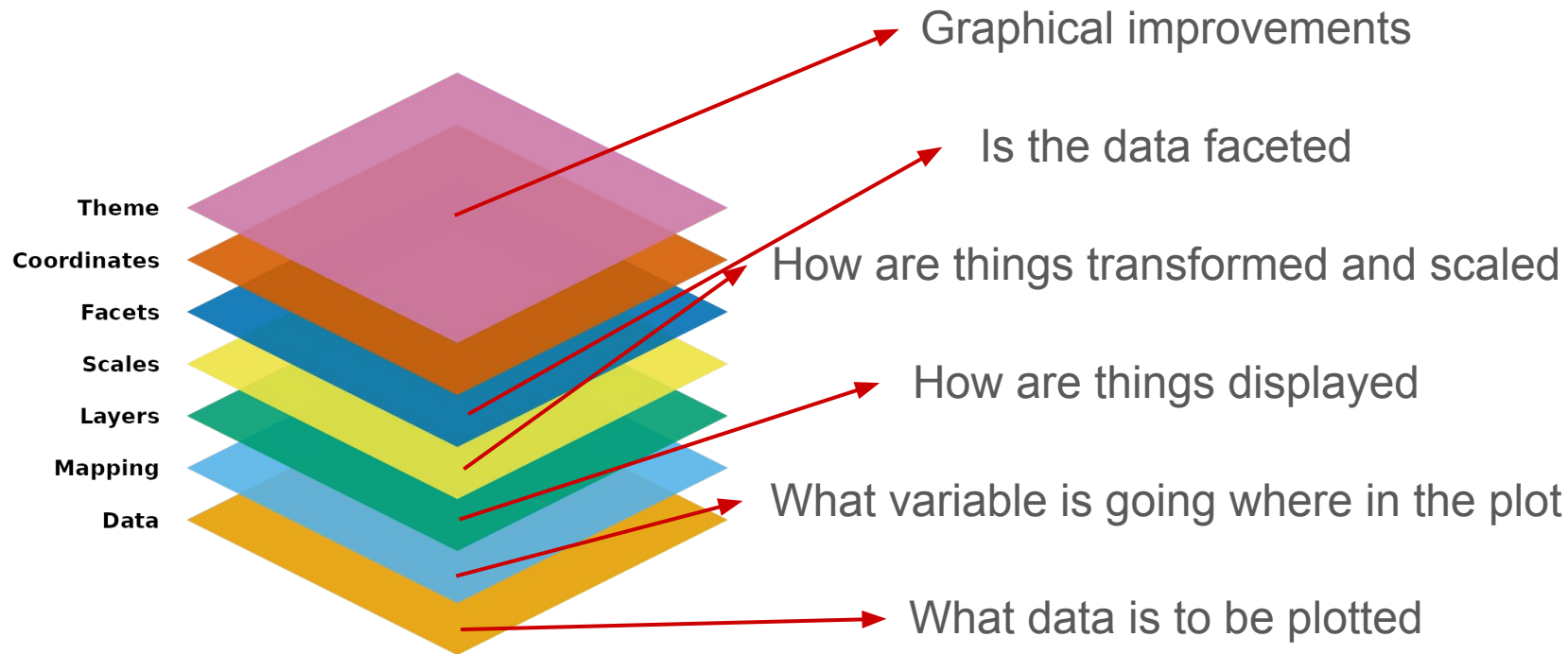
# ggplot2 - A grammar for

Theming in ggplot is about the looks of anything:  
lines, font sizes, legend positions ...

```
ggplot(df_sub,
  aes(abdominal_diameter, bmi,
      color= diagnosis))+
  geom_point()+
  scale_color_manual(
    values = c("red", "grey", "blue"))
  theme(
    legend.position = "bottom",
    axis.line = element_line(linewidth = 0.75),
    axis.line.x =
      element_line(color = "blue")
    axis.text.x = element_text(angle = 45,
                                hjust = 1)
```



# ggplot2 - A grammar for graphics



# Brief Excursion: Types of data

## Discrete data

- categorical data
- ordinal data

## Continuous data

- count data
- continuous data

→ For Plotting, we ask if what we want to map is continuous or discrete

# geom\_point()

Can be used for continuous or continuous/discrete data

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi,  
    color= diagnosis))+  
  geom_point()  
  
ggplot(df_sub,  
  aes(abdominal_diameter, bmi,  
    color = diagnosis, shape = diagnosis,  
    size = age))+  
  geom_point()
```

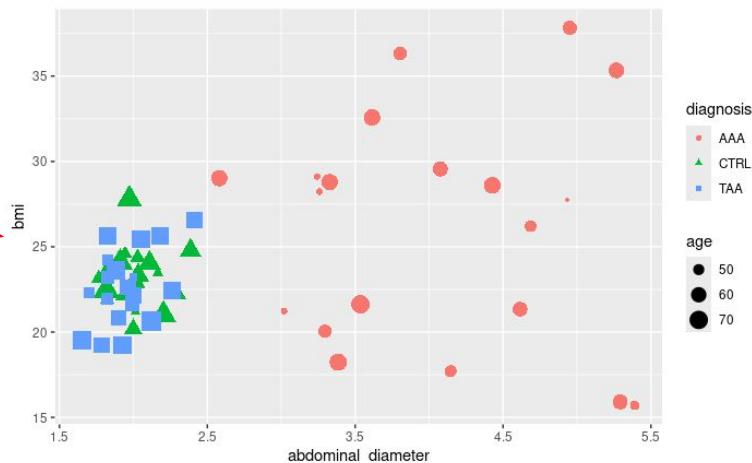
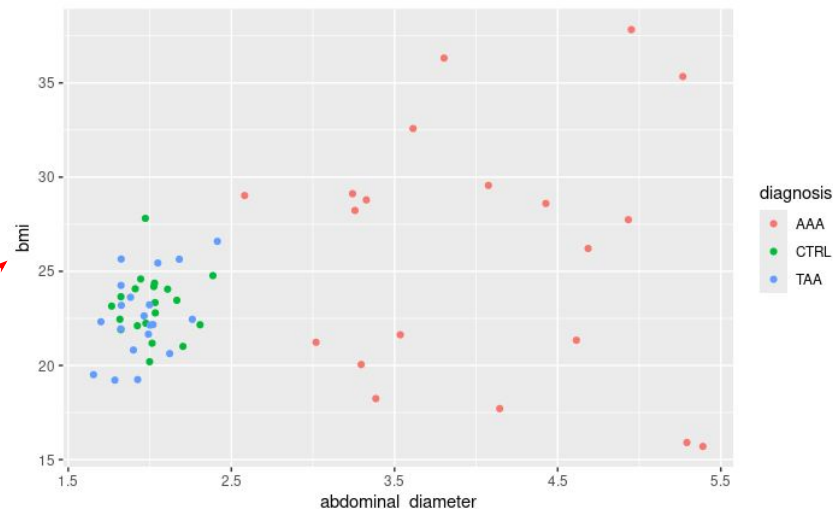
Passing the data  
mapping the data, abdominal diameter to X,  
bmi to Y and color to diagnosis  
creating a scatterplot

Passing the data  
mapping the data, abdominal diameter to X,  
bmi to Y and color and shape to diagnosis, size  
to age  
creating a scatterplot



# geom\_point()

```
ggplot(df_sub,  
  aes(abdominal_diameter, bmi,  
    color= diagnosis))+  
  geom_point()  
  
ggplot(df_sub,  
  aes(abdominal_diameter, bmi,  
    color = diagnosis, shape = diagnosis,  
    size = age))+  
  geom_point()
```



# geom\_text() and geom\_text\_repel()

Most commonly used as a accessory to geom\_point to label certain observations

```
df_sub %>%  
  mutate(patient = paste0("pat_", 1:60)) %>%  
  ggplot(aes(abdominal_diameter, bmi, color =  
diagnosis))+  
  geom_point()+  
  geom_text(aes(label = patient))
```

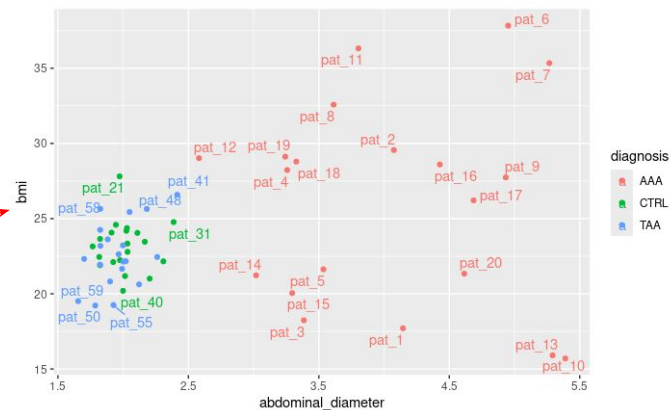
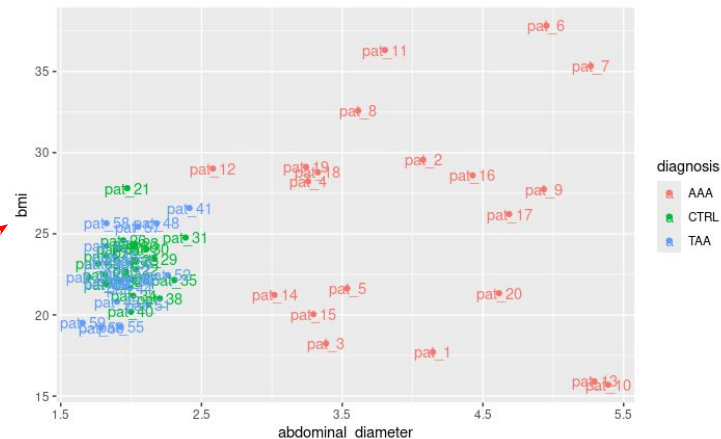
Passing the data  
adding patient variable  
mapping the data, abdominal diameter to X,  
bmi to Y and color to diagnosis  
creating a scatterplot  
adding the labels

```
df_sub %>%  
  mutate(patient = paste0("pat_", 1:60)) %>%  
  ggplot(aes(abdominal_diameter, bmi, color =  
diagnosis))+  
  geom_point()+  
  ggrepel::geom_text_repel(aes(label = patient))
```

Passing the data  
adding patient variable  
mapping the data, abdominal diameter to X,  
bmi to Y and color to diagnosis  
creating a scatterplot  
adding the labels using ggrepel

# geom\_text() and geom\_text\_repel()

```
df_sub %>%  
  mutate(patient = paste0("pat_", 1:60)) %>%  
  ggplot(aes(abdominal_diameter, bmi, color =  
diagnosis)) +  
    geom_point() +  
    geom_text(aes(label = patient))  
  
df_sub %>%  
  mutate(patient = paste0("pat_", 1:60)) %>%  
  ggplot(aes(abdominal_diameter, bmi, color =  
diagnosis)) +  
    geom_point() +  
    ggrepel::geom_text_repel(aes(label = patient))
```



# geom\_text() and geom\_text\_repel()

```
df_sub %>%  
  mutate(patient = paste0("pat_", 1:60)) %>%  
  ggplot(aes(abdominal_diameter, bmi,  
    color = diagnosis))+  
  geom_point()+  
  ggrepel::geom_text_repel(  
    data = . %>% slice_max(abdominal_diameter,  
                          n = 5),  
    label = patient))
```

```
df_sub %>%  
  mutate(patient = paste0("pat_", 1:60)) %>%  
  ggplot(aes(abdominal_diameter, bmi,  
    color = diagnosis))+  
  geom_point()+  
  ggrepel::geom_text_repel(  
    data = . %>% filter(diagnosis == "AAA"),  
    label = patient))
```

Passing the data  
adding patient variable  
mapping the data, abdominal diameter to X,  
bmi to Y and color to diagnosis  
creating a scatterplot  
adding the labels using ggrepel  
subsetting the data argument for just  
*geom\_text\_repel()* using *slice\_max()*  
specifying the label aesthetic

Passing the data  
adding patient variable  
mapping the data, abdominal diameter to X,  
bmi to Y and color to diagnosis  
creating a scatterplot  
adding the labels using ggrepel  
subsetting the data argument using *filter()*  
specifying the label aesthetic

# geom\_text() and geom\_text\_repel()

```
df_sub %>%  
  mutate(patient = paste0("pat_", 1:60)) %>%  
  ggplot(aes(abdominal_diameter, bmi,  
    color = diagnosis))+  
  geom_point()+  
  ggrepel::geom_text_repel(  
    data = . %>% slice_max(abdominal_diameter,  
                           n = 5),  
    label = patient))
```

```
df_sub %>%  
  mutate(patient = paste0("pat_", 1:60)) %>%  
  ggplot(aes(abdominal_diameter, bmi,  
    color = diagnosis))+  
  geom_point()+  
  ggrepel::geom_text_repel(  
    data = . %>% filter(diagnosis == "AAA"),  
    label = patient))
```

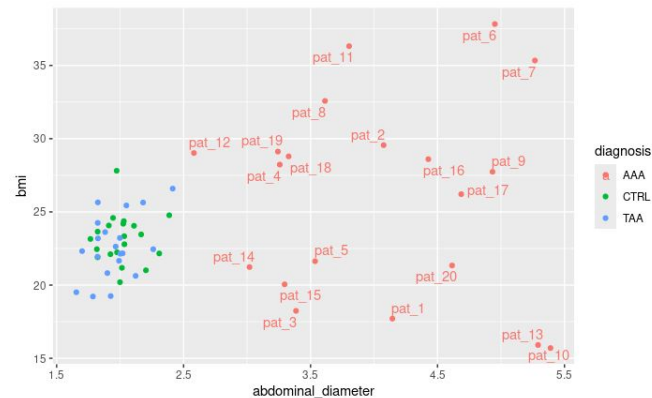
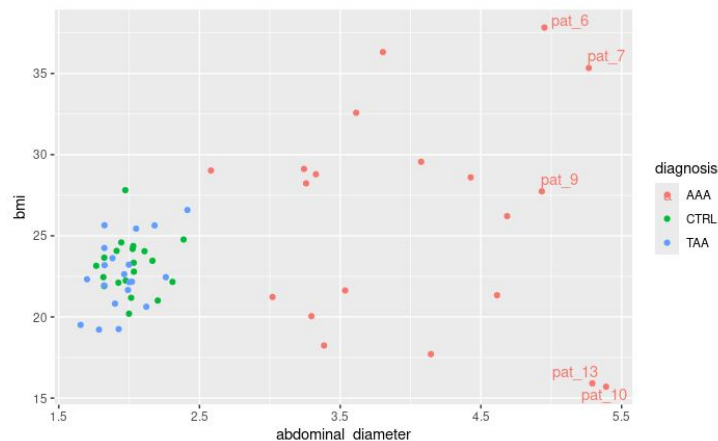
Passing the data  
adding patient variable  
mapping the data, abdominal diameter to X,  
bmi to Y and color to diagnosis  
creating a scatterplot  
adding the labels using ggrepel  
subsetting the data argument for just  
*geom\_text\_repel()* using *slice\_max()*  
specifying the label aesthetic

Passing the data  
adding patient variable  
mapping the data, abdominal diameter to X,  
bmi to Y and color to diagnosis  
creating a scatterplot  
adding the labels using ggrepel  
subsetting the data argument using *filter()*  
specifying the label aesthetic

# geom\_text() and geom\_text\_repel()

```
df_sub %>%  
  mutate(patient = paste0("pat_", 1:60)) %>%  
  ggplot(aes(abdominal_diameter, bmi,  
    color = diagnosis))+  
  geom_point()+  
  ggrepel::geom_text_repel(  
    data = . %>% slice_max(abdominal_diameter,  
                          n = 5),  
    label = patient))
```

```
df_sub %>%  
  mutate(patient = paste0("pat_", 1:60)) %>%  
  ggplot(aes(abdominal_diameter, bmi,  
    color = diagnosis))+  
  geom_point()+  
  ggrepel::geom_text_repel(  
    data = . %>% filter(diagnosis == "AAA"),  
    label = patient))
```



# geom\_boxplot() + geom\_violin ()

Requires discrete and continuous data. Can be combined with geom\_point

```
ggplot(df_sub,  
  aes(diagnosis, abdominal_diameter))+  
  geom_boxplot()
```

```
ggplot(df_sub,  
  aes(diagnosis, abdominal_diameter  
    color = diagnosis))+  
  geom_boxplot()+  
  geom_point()
```

```
ggplot(df_sub,  
  aes(diagnosis, abdominal_diameter))+  
  geom_violin()
```

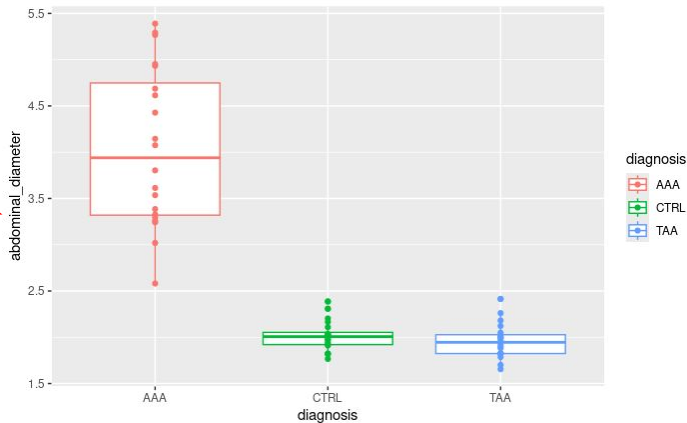
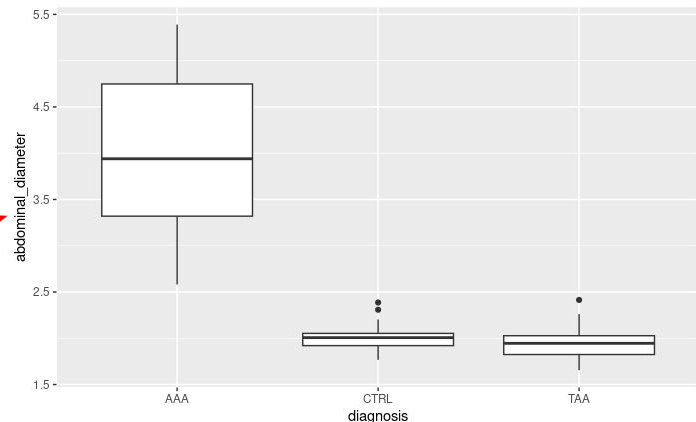
Passing the data  
mapping the data, abdominal diameter to X,  
bmi to Y and color to diagnosis  
creating a scatterplot

# geom\_boxplot() + geom\_violin ()

```
ggplot(df_sub,  
  aes(diagnosis, abdominal_diameter))+  
  geom_boxplot()
```

```
ggplot(df_sub,  
  aes(diagnosis, abdominal_diameter  
    color = diagnosis))+  
  geom_boxplot()+  
  geom_point()
```

```
ggplot(df_sub,  
  aes(diagnosis, abdominal_diameter))+  
  geom_violin()
```



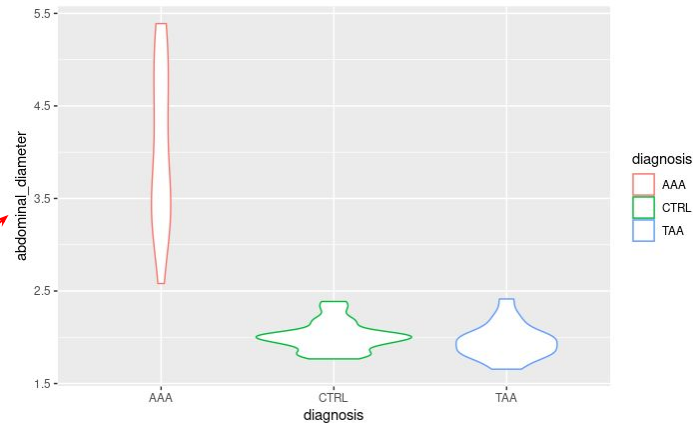


# geom\_boxplot() + geom\_violin ()

```
ggplot(df_sub,  
  aes(diagnosis, abdominal_diameter))+  
  geom_boxplot()
```

```
ggplot(df_sub,  
  aes(diagnosis, abdominal_diameter  
    color = diagnosis))+  
  geom_boxplot()+  
  geom_point()
```

```
ggplot(df_sub,  
  aes(diagnosis, abdominal_diameter))+  
  geom_violin()
```



# geom\_bar() & geom\_col()

Requires discrete and continuous data. Most commonly used to display quantities. `geom_bar()` counts things automatically, `geom_col()` displays values from the data.

```
df_sub %>%  
  filter(age < 75) %>%  
  ggplot(aes(diagnosis))+  
  geom_bar()  
  
df_sub %>%  
  filter(age < 75) %>%  
  count(diagnosis) %>%  
  ggplot(aes(diagnosis, n))+  
  geom_col()
```

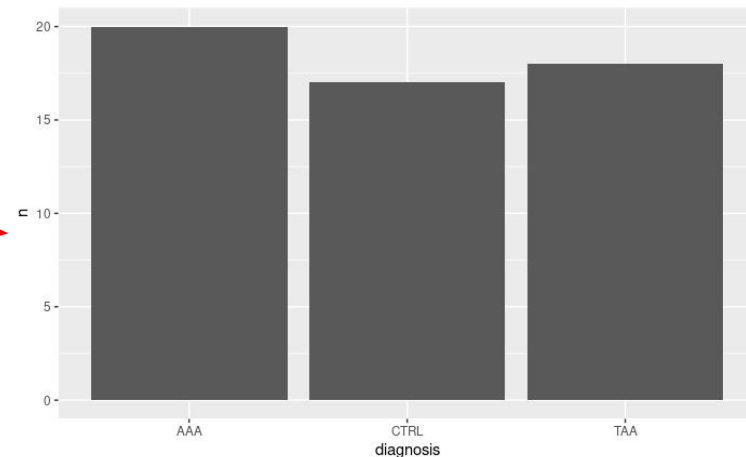
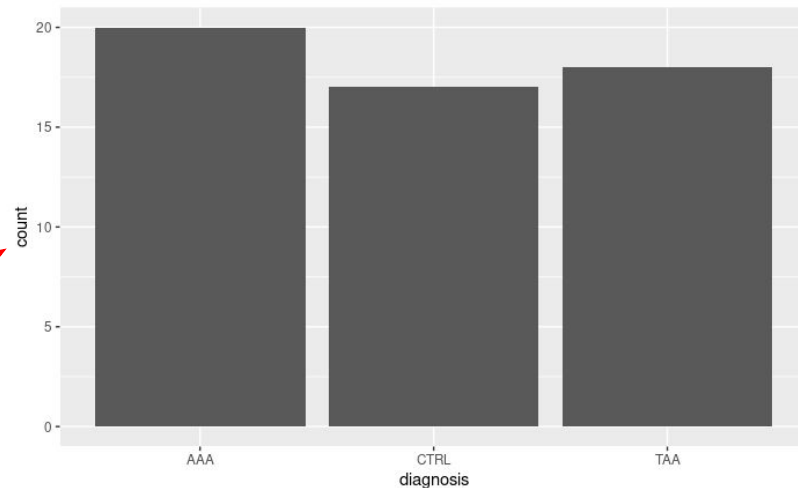
Passing the data  
filtering by age  
mapping the data, only X or Y can be given, the other is the count  
creating a bar plot

Passing the data  
filtering by age  
counting the different diagnoses  
mapping the data, diagnosis to x and the count to y  
creating a bar plot

# geom\_bar() & geom\_col()

```
df_sub %>%  
  filter(age < 75) %>%  
  ggplot(aes(diagnosis))+  
  geom_bar()
```

```
df_sub %>%  
  filter(age < 75) %>%  
  count(diagnosis) %>%  
  ggplot(aes(diagnosis, n))+  
  geom_col()
```



# geom\_bar() - stack, dodge and fill

```
df_plot <- filter(df_sub, bmi > 20) %>%  
  mutate(old = ifelse(age > 75, "older  
patient", "younger patient"))
```

Filtering the df by bmi  
creating an age classifier

```
df_plot %>%  
  ggplot(aes(diagnosis, fill = old))+  
  geom_bar(position = "stack")
```

passing the data  
mapping the data, x to diagnosis, fill to classifier  
creating a bar plot, stacked

```
df_plot %>%  
  ggplot(aes(diagnosis, fill = old))+  
  geom_bar(position = "dodge")
```

passing the data  
mapping the data, x to diagnosis, fill to classifier  
creating a bar plot, stacked

```
df_plot%>%  
  ggplot(aes(diagnosis, fill = old))+  
  geom_bar(position = "fill")
```

passing the data  
mapping the data, x to diagnosis, fill to classifier  
creating a bar plot, stacked

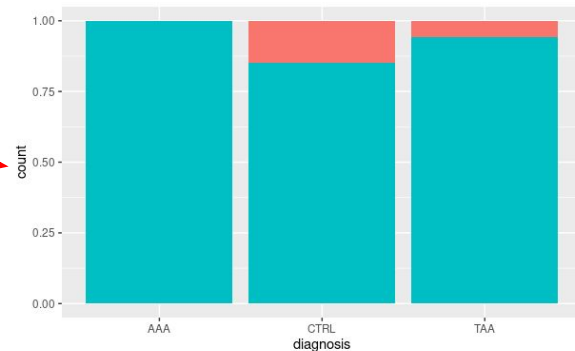
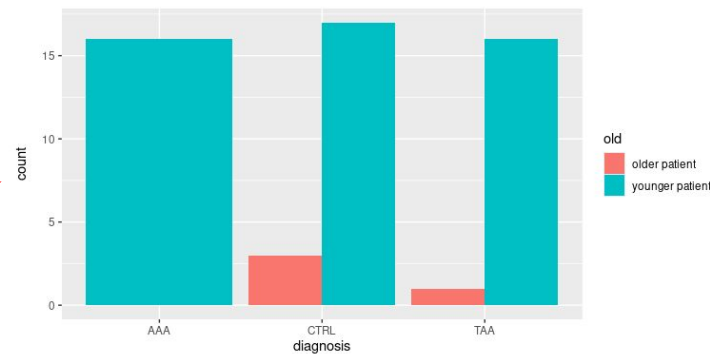
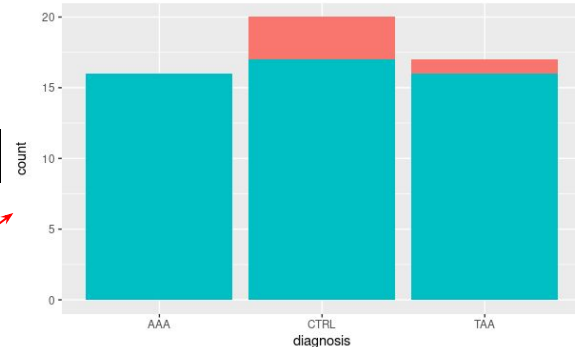
# geom\_bar() - stack, dodge and fill

```
df_plot <- filter(df_sub, bmi > 20) %>%  
  mutate(old = ifelse(age > 75, "older  
patient", "younger patient"))
```

```
df_plot %>%  
  ggplot(aes(diagnosis, fill = old))+  
  geom_bar(position = "stack")
```

```
df_plot %>%  
  ggplot(aes(diagnosis, fill = old))+  
  geom_bar(position = "dodge")
```

```
df_plot%>%  
  ggplot(aes(diagnosis, fill = old))+  
  geom_bar(position = "fill")
```



# Special case: geom\_histogram() and geom\_density()

Plots the distribution of one variable as a histogram

```
df_sub %>%  
  ggplot(aes(bmi))+  
  geom_histogram()
```

Passing the data  
mapping x to BMI  
plotting the histogram

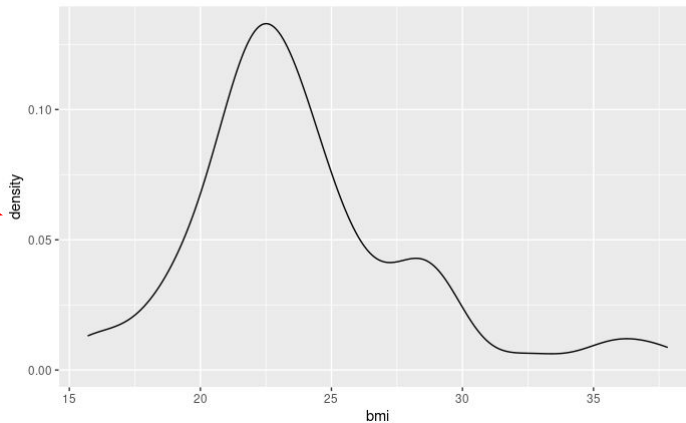
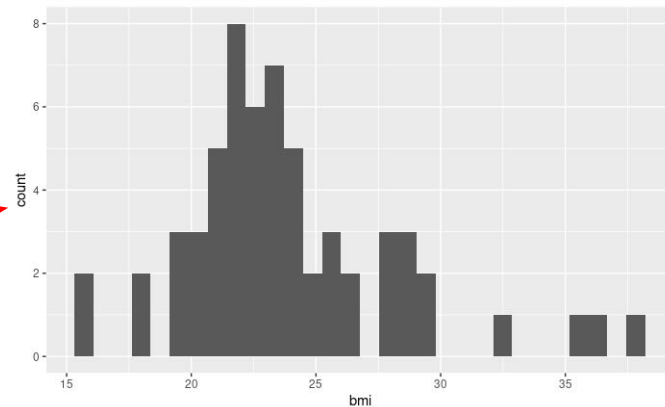
```
df_sub %>%  
  ggplot(aes(bmi))+  
  geom_density()
```

Passing the data  
mapping x to BMI  
plotting the density plot

# Special case: geom\_histogram() and geom\_density()

```
df_sub %>%  
  ggplot(aes(bmi))+  
  geom_histogram()
```

```
df_sub %>%  
  ggplot(aes(bmi))+  
  geom_density()
```



# stat\_summary()

Special case for visualisations: takes values and calculates summary statistics to display. If you want a bar chart displaying the mean per group, you would use `stat_summary()` and specify the geom.

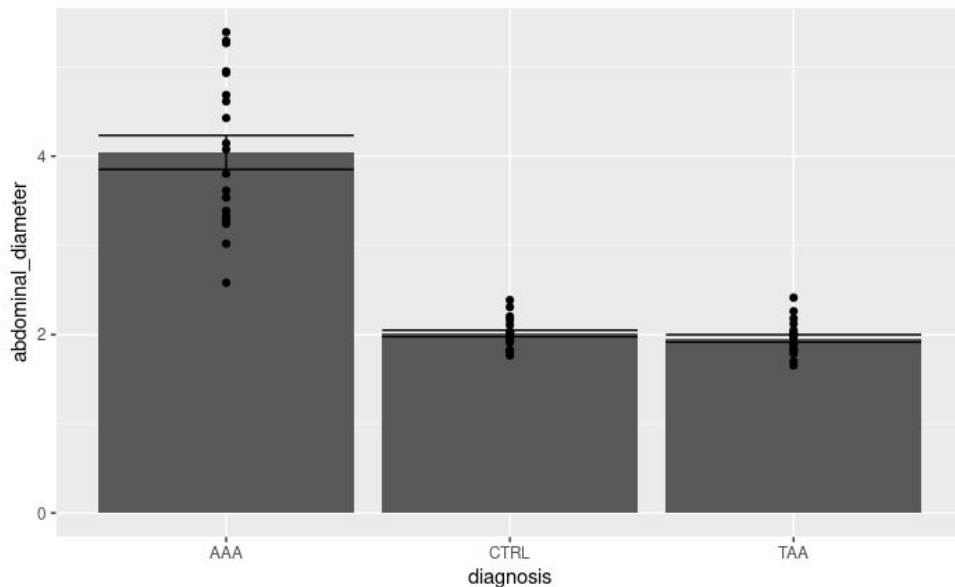
```
df_sub %>%  
  ggplot(aes(diagnosis,  
             abdominal_diameter))+  
  stat_summary(geom = "bar")+  
  stat_summary(geom = "errorbar")+  
  geom_point()
```

Passing the data  
filtering by age  
mapping the data, diagnosis to x and the diameter to y  
creating a bar plot, calculating the mean (and SE) by default  
getting the SE for errorbars  
adding individual observations



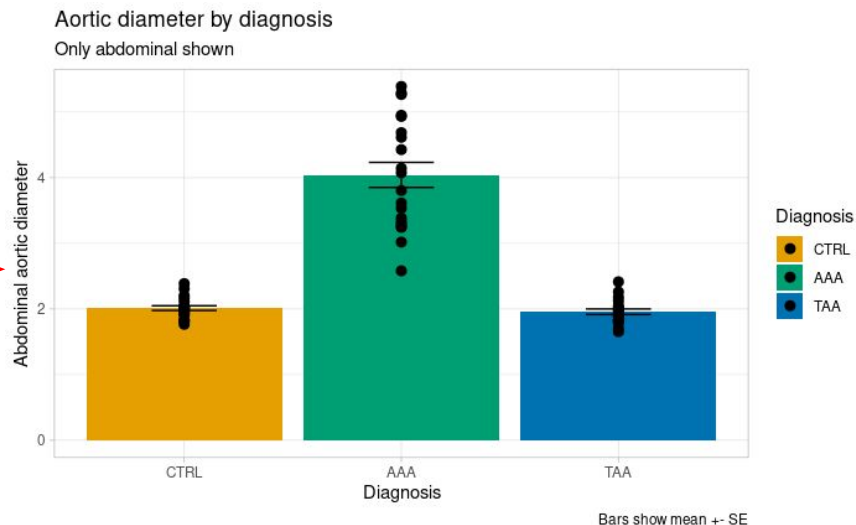
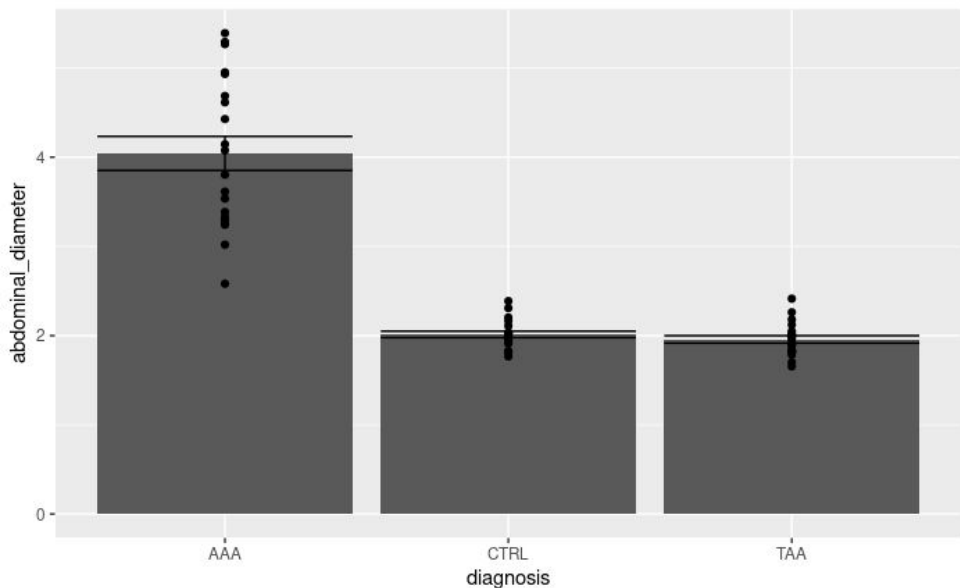
# stat\_summary()

```
df_sub %>%  
  ggplot(aes(diagnosis,  
             abdominal_diameter))+  
  stat_summary(geom = "bar")+  
  stat_summary(geom = "errorbar")+  
  geom_point()
```



# Building plots - data to visuals

Without changing the way data or visualisation, lots can be done to improve the visuals of a plot.



# geom\_smooth()

Special case for visualisations: takes values and calculates summary statistics to display. If you want a bar chart displaying the mean per group, you would use `stat_summary()` and specify the geom.

```
df_sub %>%  
  ggplot()
```

## Still needs work

```
  stat_summary(geom = "bar")+  
  stat_summary(geom = "errorbar")+  
  geom_point()
```

creating a bar plot, calculating the mean (and SE) by default  
getting the SE for errorbars  
adding individual observations

# labs()

By default, the axes and other aesthetics are labeled by their column names. Usually, you want to change that. `labs()` allows you to specify a label to modify.

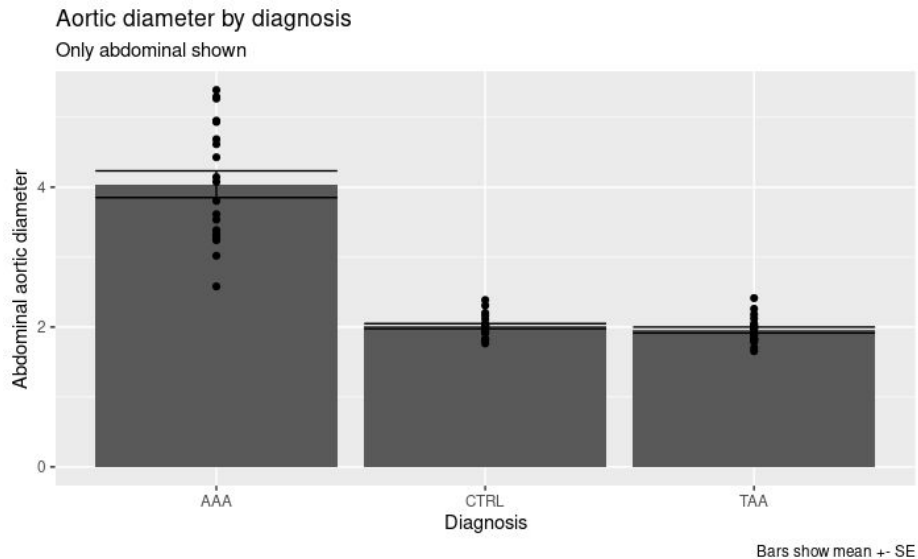
```
df_sub %>%  
  ggplot(aes(diagnosis,  
             abdominal_diamter))+  
  stat_summary(geom = "bar")+  
  stat_summary(geom = "errorbar")+  
  geom_point()+  
  labs(  
    x = "Diagnosis",  
    y = "Abdominal aortic diameter",  
    title = "Aortic diameter by  
           diagnosis",  
    subtitle = "Only abdominal shown",  
    caption = "Bars show mean +- SE")
```

Passing the data  
filtering by age  
mapping the data, diagnosis to x and the diameter to y  
creating a bar plot, calculating the mean (and SE) by default  
getting the SE for errorbars  
adding individual observations  
adding `labs()` function  
x axis label  
y axis label  
title for the plot

Adding a subtitle  
Adding a caption

# labs()

```
df_sub %>%  
  ggplot(aes(diagnosis,  
             abdominal_diameter))+  
  stat_summary(geom = "bar")+  
  stat_summary(geom = "errorbar")+  
  geom_point()+  
  labs(  
    x = "Diagnosis",  
    y = "Abdominal aortic diameter",  
    title = "Aortic diameter by  
           diagnosis",  
    subtitle = "Only abdominal shown",  
    caption = "Bars show mean +/- SE")
```





# scales package

An [R package](#) designed for the work with scales within ggplot:

- contains large amount of utility functions,:
  - `label_percent()` for percentage scales
  - `label_scientific()` for scientific notation

```
df_plot%>%  
  ggplot(aes(diagnosis, fill = old))+  
  geom_bar(position = "fill")
```

passing data  
mapping  
building a filled bar chart

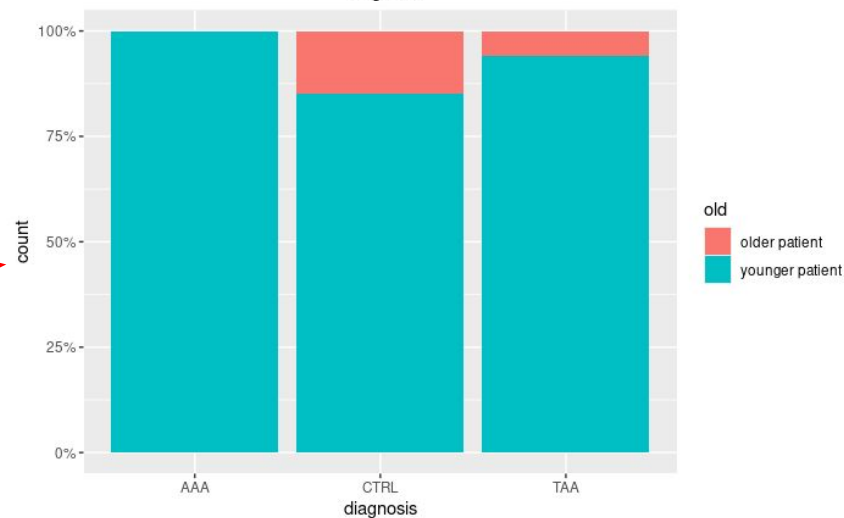
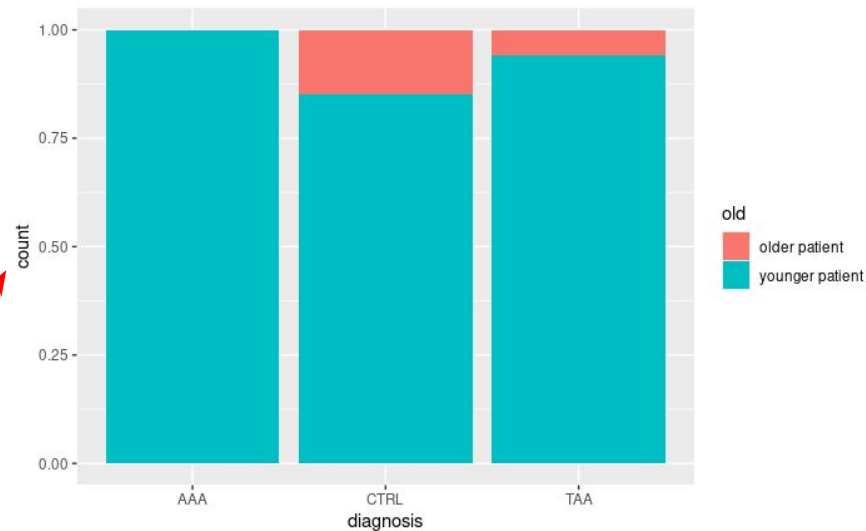
```
df_plot%>%  
  ggplot(aes(diagnosis, fill = old))+  
  geom_bar(position = "fill")+  
  scale_y_continuous(labels =  
scales::label_percent())
```

adjusting the labels for the y axis

# scales package

```
df_plot%>%  
  ggplot(aes(diagnosis, fill = old))+  
  geom_bar(position = "fill")
```

```
df_plot%>%  
  ggplot(aes(diagnosis, fill = old))+  
  geom_bar(position = "fill")+  
  scale_y_continuous(labels =  
scales::label_percent())
```



# theme\_X - prebuilt themes

In ggplot, there are a number of [prebuilt themes](#):

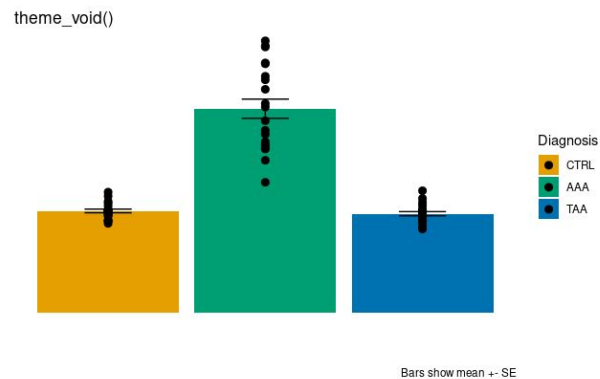
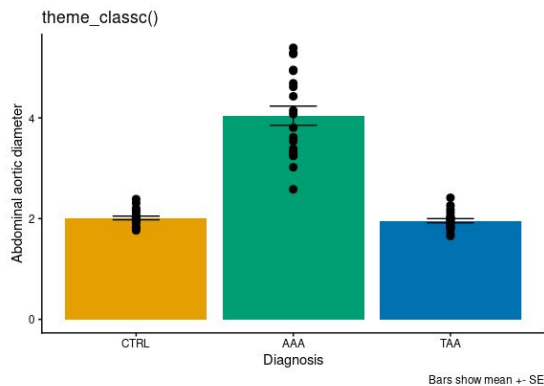
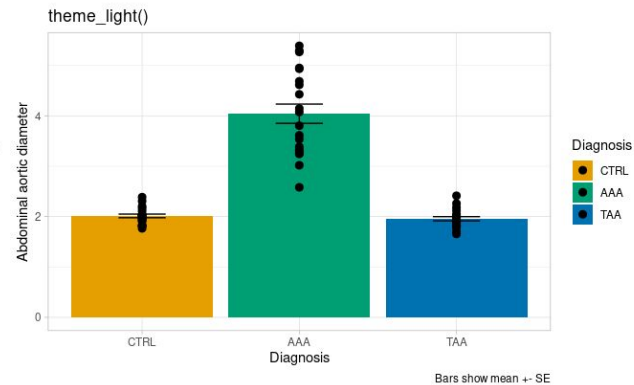
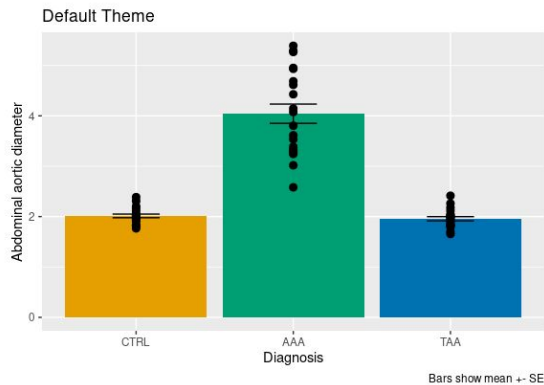
- `theme_grey()` is the default, grey background, white lines
- `theme_bw()` for heavy contrast
- `theme_light()` lighter version of the heavy contrast theme
- `theme_minimal()` minimal annotations and lines
- `theme_classic()` for clear axes and no grid
- `theme_void()` removes everything that is not the data

Important functions

- `theme_set()`
- `theme_minimal(base_size = 12)`



# Showcase of different themes



# annotate, geom\_hline(), geom\_vline(), geom\_abline()

In building plots, you sometimes want reference lines, annotations or highlight a certain area.

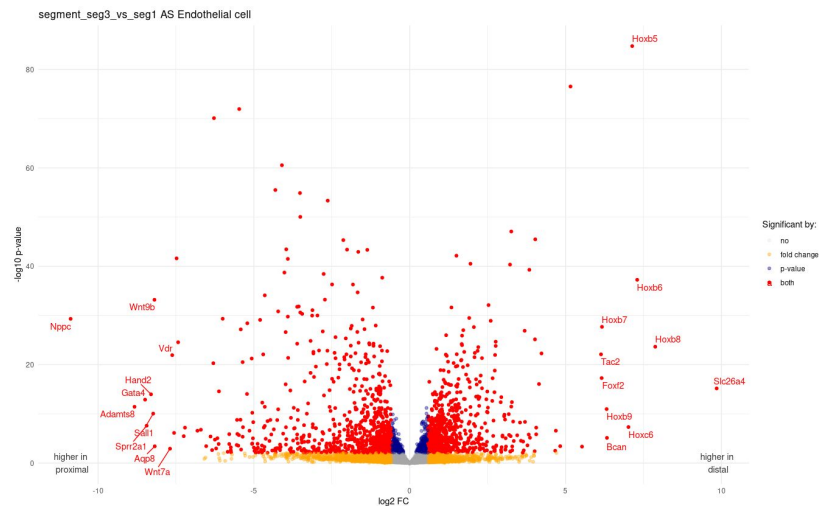
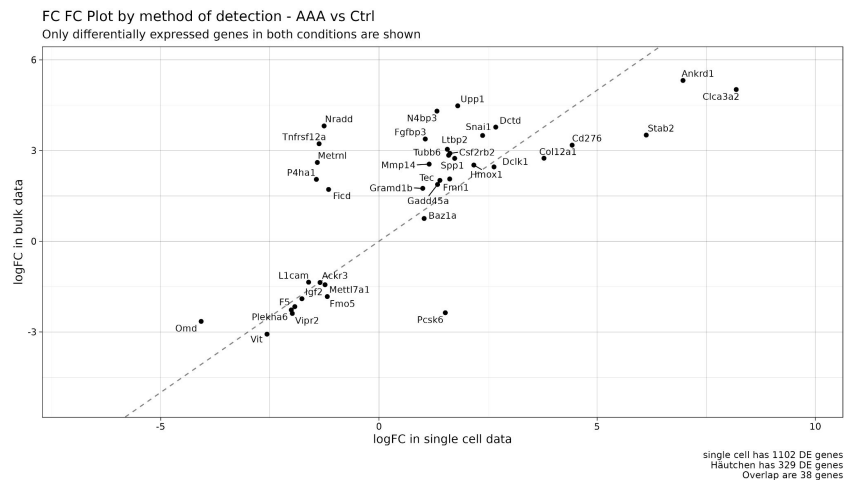
geom\_hline() adds a horizontal line

geom\_vline() adds vertical line

geom\_abline() adds a line with an intersect and a slope

annotate() can add a single element of any geom you choose, such as geom\_text() or geom\_rect()

# Little annotations make plots more readable





# forcats package

A [R package](#) for working with factors.

As you will learn, ggplot converts a lot of things to factors under the hood, and uses the levels of these factors to order things like our diagnosis variable.

One of the two functions below is used in almost any plot I create:

- `fct_relevel()` - manually reorder the levels of a factor
- `fct_reorder()` - order the levels of one factor by some numeric variable

# fct\_relevel()



```
df_sub %>%  
  ggplot(  
    aes(diagnosis, abdominal_diameter,  
        color = diagnosis))+  
  geom_boxplot()+  
  scale_color_manual(values =  
    c("red", "grey", "blue"))
```

```
df_sub %>%  
  mutate(  
    diagnosis = fct_relevel(diagnosis,  
                           "CTRL")) %>%  
  ggplot(  
    aes(diagnosis, abdominal_diameter,  
        color = diagnosis))+  
  geom_boxplot()+  
  scale_color_manual(values =  
    c("red", "grey", "blue"))
```

passing the data

plotting

mapping diagnosis to x, diameter to y, color to diagnosis

making a boxplot

setting the colors

to red, grey, blue

passing the data

making new columns

replacing the existing diagnosis column with a factor version; the reference level for the factor will be CTRL

plotting

mapping diagnosis to x, diameter to y, color to diagnosis

making a boxplot

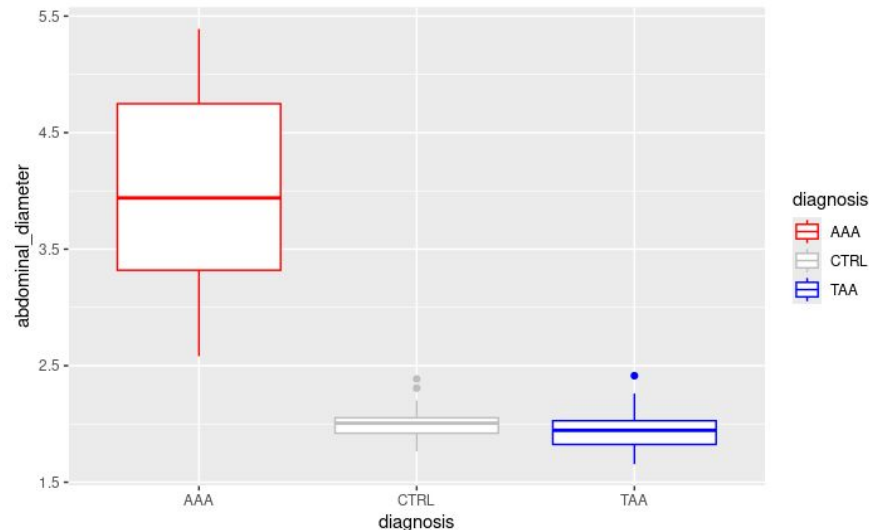
setting the colors

to red, grey, blue

# fct\_relevel()



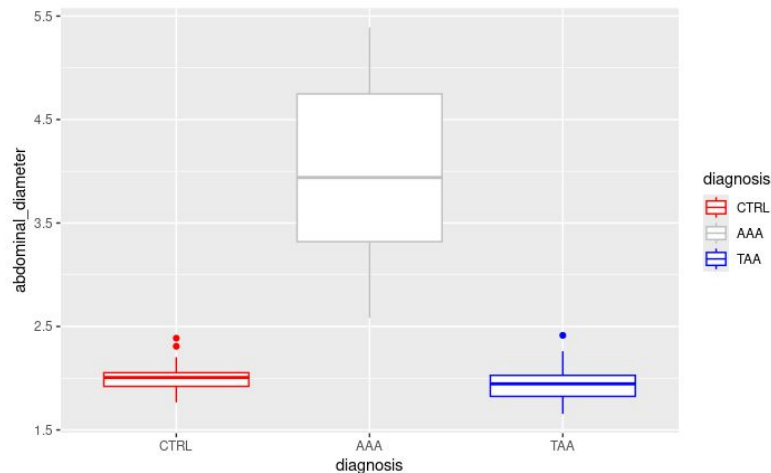
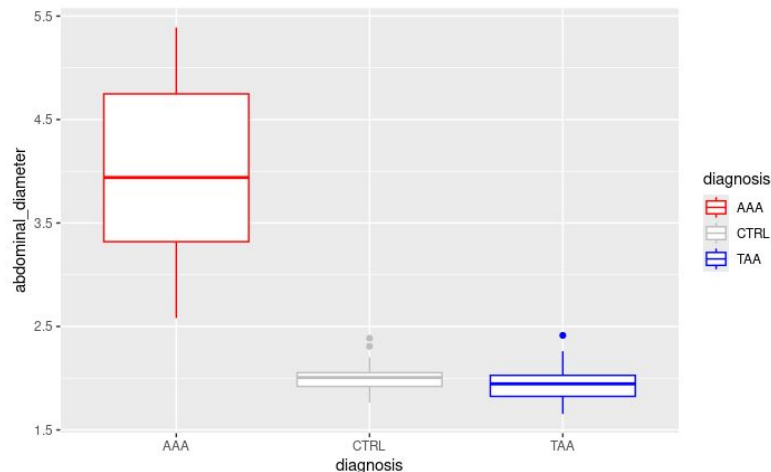
```
df_sub %>%  
  ggplot(  
    aes(diagnosis, abdominal_diameter,  
        color = diagnosis))+  
  geom_boxplot()+  
  scale_color_manual(values =  
    c("red", "grey", "blue"))  
  
df_sub %>%  
  mutate(  
    diagnosis = fct_relevel(diagnosis,  
                           "CTRL")) %>%  
  ggplot(  
    aes(diagnosis, abdominal_diameter,  
        color = diagnosis))+  
  geom_boxplot()+  
  scale_color_manual(values =  
    c("red", "grey", "blue"))
```



# fct\_relevel()

```
df_sub %>%  
  ggplot(  
    aes(diagnosis, abdominal_diameter,  
        color = diagnosis))+  
  geom_boxplot()+  
  scale_color_manual(values =  
    c("red", "grey", "blue"))
```

```
df_sub %>%  
  mutate(  
    diagnosis = fct_relevel(diagnosis,  
                            "CTRL")) %>%  
  ggplot(  
    aes(diagnosis, abdominal_diameter,  
        color = diagnosis))+  
  geom_boxplot()+  
  scale_color_manual(values =  
    c("red", "grey", "blue"))
```



# fct\_relevel()



```
df_sub %>%  
  ggplot(  
    aes(diagnosis, abdominal_diameter,  
        color = diagnosis))+  
  geom_boxplot()+  
  scale_color_manual(values =  
    c("red", "grey", "blue"))  
  
df_sub %>%  
  mutate(  
    diagnosis = fct_reorder(diagnosis,  
                           abdominal_diameter)) %>%  
  ggplot(  
    aes(diagnosis, abdominal_diameter,  
        color = diagnosis))+  
  geom_boxplot()+  
  scale_color_manual(values =  
    c("red", "grey", "blue"))
```

passing the data

plotting

mapping diagnosis to x, diameter to y, color to diagnosis

making a boxplot

setting the colors

to red, grey, blue

passing the data

making new columns

replacing the existing diagnosis column with a factor version; the reference level for the factor will be CTRL

plotting

mapping diagnosis to x, diameter to y, color to diagnosis

making a boxplot

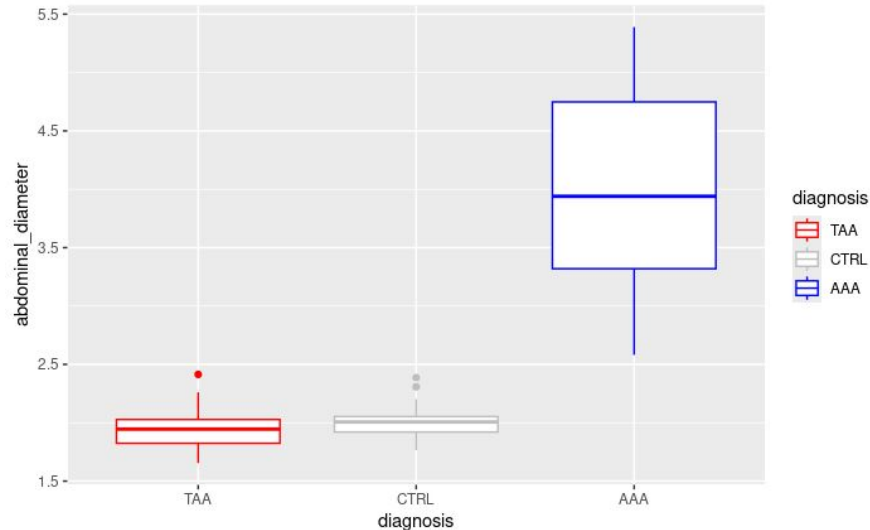
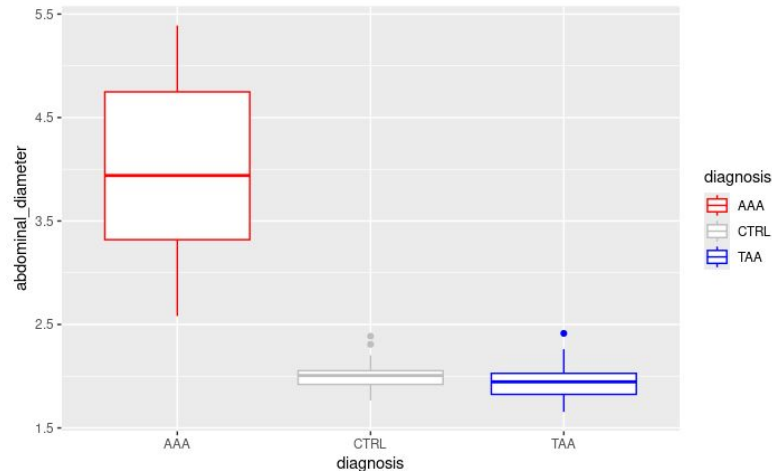
setting the colors

to red, grey, blue



# fct\_reorder()

```
df_sub %>%  
  ggplot(  
    aes(diagnosis, abdominal_diameter,  
        color = diagnosis))+  
  geom_boxplot()+  
  scale_color_manual(values =  
    c("red", "grey", "blue"))  
  
df_sub %>%  
  mutate(  
    diagnosis = fct_reorder(diagnosis,  
                            abdomninal_reorder)) %>%  
  ggplot(  
    aes(diagnosis, abdominal_diameter,  
        color = diagnosis))+  
  geom_boxplot()+  
  scale_color_manual(values =  
    c("red", "grey", "blue"))
```



# Practise Time!

## Tasks

### Easy

E1: Subset the dataset to include only contain 3 observations per diagnosis. Create a box plot of aortic diameter. Repeat the sub-setting and the plotting. Is the plot changing?

E2: Repeat this with 10 observations per diagnosis. How stable is the re-sampling then?

E3: Lets stick with the 10 observations per observations. Create a boxplot again. Add points on top of it.

E4: What is the difference between `fill` and `color` for `geom_boxplot`? Try it out by adding some color to your boxplots.

E4: Create a scatter plot using BMI and abdominal aortic diameter. Do you think there is connection?

E5: Reuse the code for the scatter plot. Set the alpha of the points to 0.5 by using `geom_point(alpha = 0.5)`. What happens? How does the plot change when you change the value?

E6: Use the same process as above to try some values for the shape of `geom_point()`. The vlaues range from 0 to 25. Which one is a square with a cross in it?

E7: Using `aes()`, set the size of the points to age. What does the plot look like?

E8: Try setting the shape of the points by age. What is going wrong?

E5: Filter the original data set to patients with a BMI larger than 20. How many patients remain per diagnosis? Plot a bar chart to visualise it.

#### Table of contents

Projects and the here  
package

ggplot2 and data  
visualisation

forcats

Mean separation

Tasks

[Easy](#)

Medium

Hard



# Statistical testing

In our work, we test 3 kinds of things:

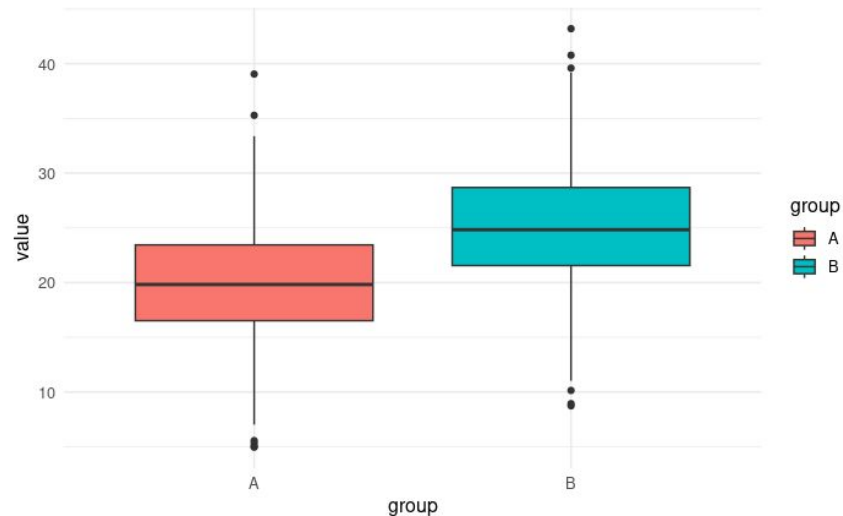
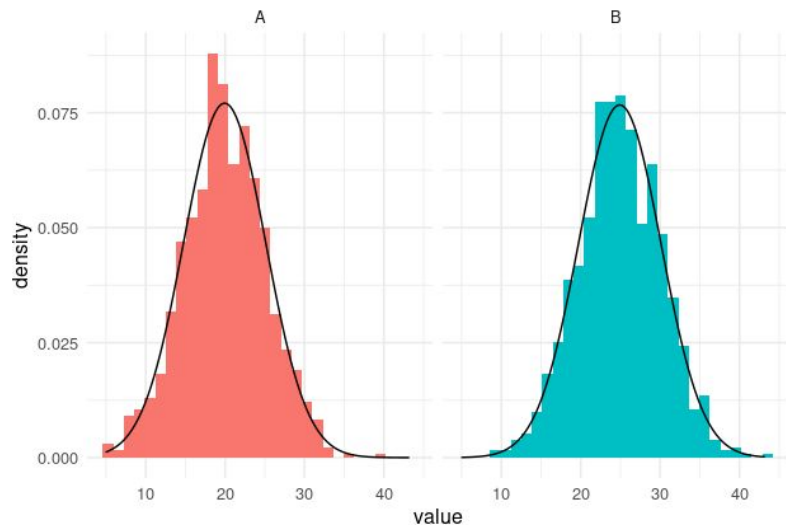
1. Mean separation: Is the mean between two groups different?
2. Over-representation: Is there more of than expected in a category?
3. Correlation: Is a change in one associated with a change in the other?

# Statistical testing

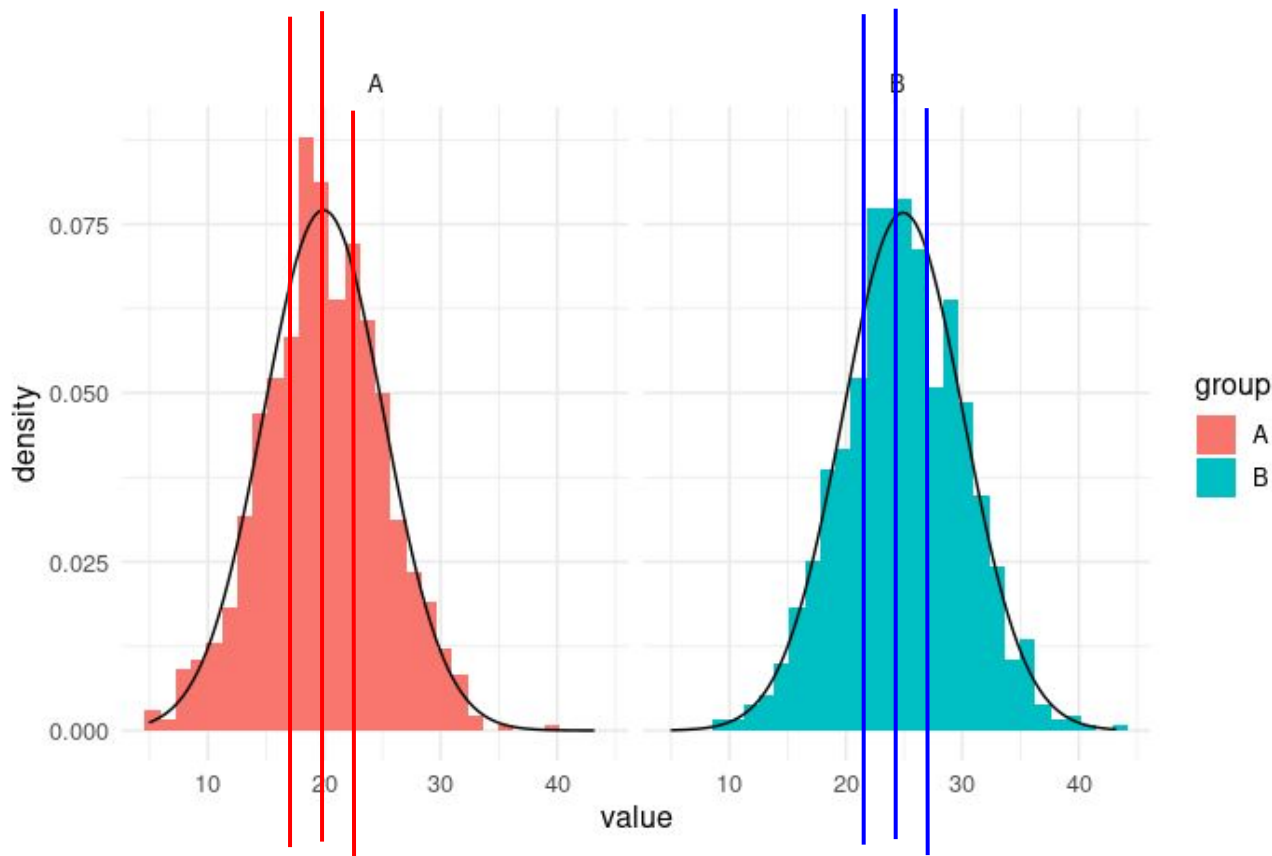
In our work, we test 3 kinds of things:

1. **Mean separation:** Is the mean between two groups different?
2. Over-representation: Is there more of than expected in a category?
3. Correlation: Is a change in one associated with a change in the other?

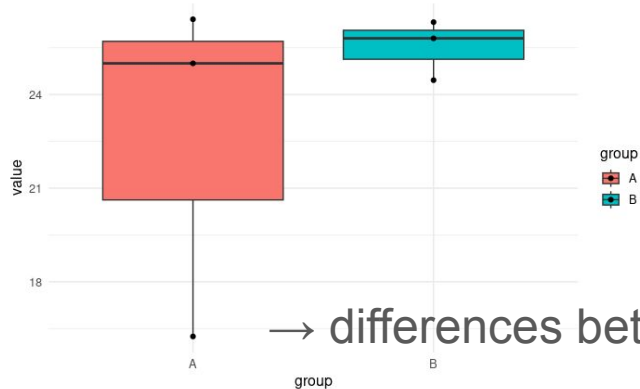
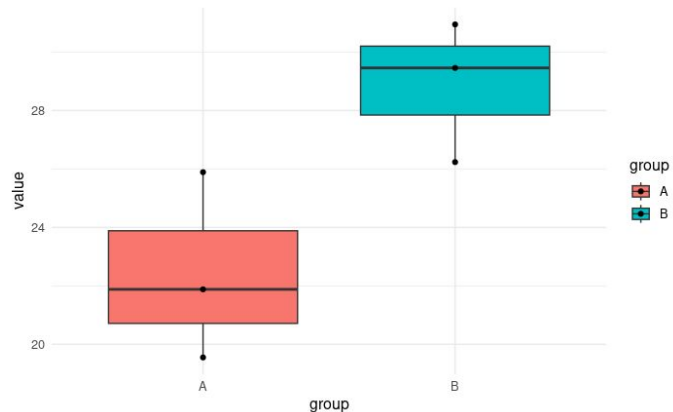
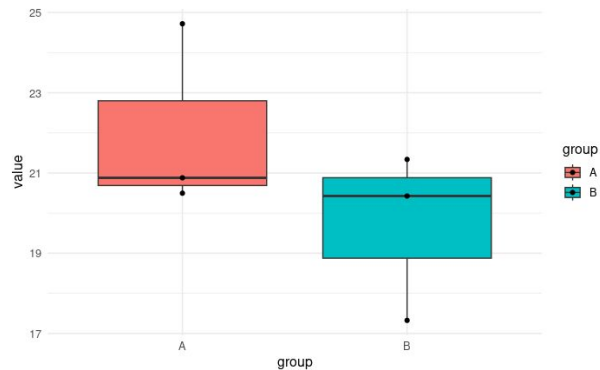
# Two groups, A and B, 1000 samples each



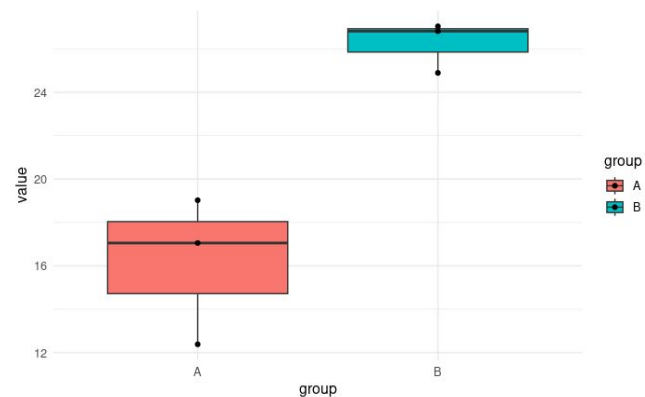
# Experiment = sampling from the true distribution



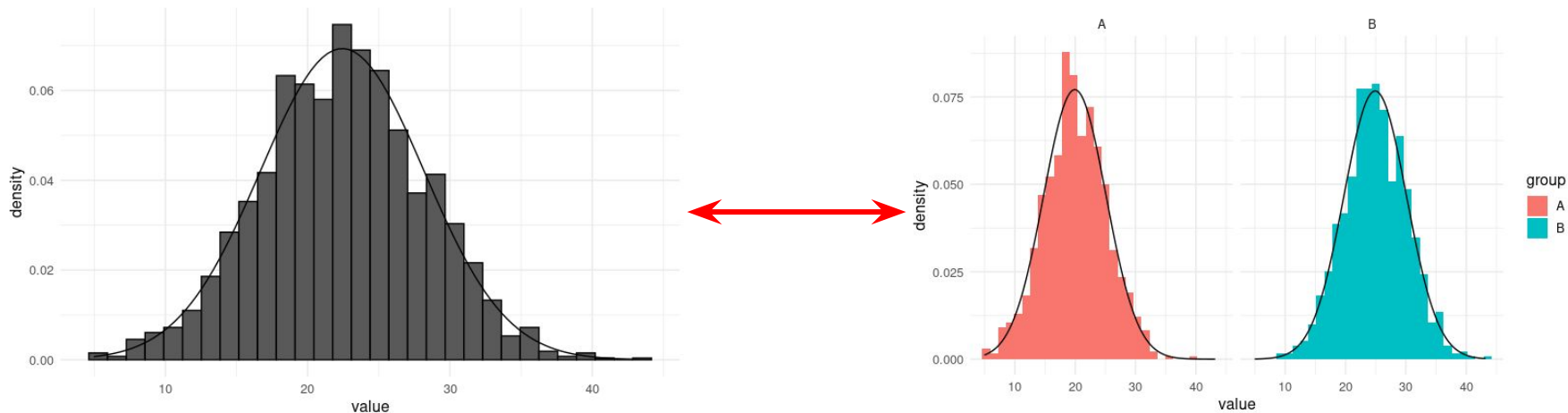
# Drawing samples (n=3)



→ differences between samples



# Are we drawing from one or two distributions?



Statistics to the rescue



# Test for significance - Three easy steps

1. Pick a test
2. Make your computer do it
3. Interpret the results

# Picking a Test

- every test has some assumptions about the data
- the more restrictive the assumptions, the “lower” the threshold for significance
- in some cases, you can take the assumptions as given (“central limit theorem”)

Assume normality

**t-Test**

ANOVA

Don't assume normality

**Mann-Whitney-U / Wilcoxon-Rank-Sum  
test**

Krushkal wallis test

# Make your computer do it

- in Prism, there is a lot of clicking
- in base R, there are several functions for it (they're all kinda bad)
- in the [rstatix package](#), they have been reimplemented
  - t-Test: `t_test(data, formula)`
  - Wilcox: `wilcox_test(data, formula)`
- formula is how you want to explain the difference, i.e.

value ~ group

thing to be explained

thing doing the explaining



# Interpret the Results

- most important bit: the p-value
- p-value is the probability to find data at least as extreme as the one used in the test under the null hypothesis (“they share one distribution/mean”)
- lower numbers means lower probability
- generally, the level of 0.05 is considered an acceptable threshold

# t\_test() in action

```
df_sub %>%  
  rstatix::t_test(  
    abdominal_diameter ~ diagnosis  
  )
```

Passing the data

calling the t test function

formula: abdominal diameter is explained by diagnosis

A tibble: 3 × 10

|   | <b>.y.</b><br><chr> | <b>group1</b><br><chr> | <b>group2</b><br><chr> | <b>n1</b><br><int> | <b>n2</b><br><int> | <b>statistic</b><br><dbl> | <b>df</b><br><dbl> | <b>p</b><br><dbl> | <b>p.adj</b><br><dbl> | <b>p.adj.signif</b><br><chr> |
|---|---------------------|------------------------|------------------------|--------------------|--------------------|---------------------------|--------------------|-------------------|-----------------------|------------------------------|
| 1 | abdominal_diameter  | AAA                    | CTRL                   | 20                 | 20                 | 10.448948                 | 20.35705           | 1.25e-09          | 2.50e-09              | ****                         |
| 2 | abdominal_diameter  | AAA                    | TAA                    | 20                 | 20                 | 10.673263                 | 20.81678           | 6.74e-10          | 2.02e-09              | ****                         |
| 3 | abdominal_diameter  | CTRL                   | TAA                    | 20                 | 20                 | 1.009025                  | 37.21380           | 3.19e-01          | 3.19e-01              | ns                           |

3 rows

# t\_test() in action

```
df_sub %>%  
  rstatix::t_test(  
    abdominal_diameter ~ diagnosis  
  )
```

```
df_sub %>%  
  group_by(hypertension) %>%  
  rstatix::t_test(  
    abdominal_diameter ~ diagnosis  
  )
```

Passing the data  
calling the t test function  
formula: abdominal diameter is explained by diagnosis

Passing the data  
grouping by hypertension → one test per level  
calling the test  
formula: abdominal diameter is explained by diagnosis

A tibble: 6 × 11

|   | hypertension<br><dbl> | .y.<br><chr>       | group1<br><chr> | group2<br><chr> | n1<br><int> | n2<br><int> | statistic<br><dbl> | df<br><dbl> | p<br><dbl> | p.adj<br><dbl> | p.adj.signif<br><chr> |
|---|-----------------------|--------------------|-----------------|-----------------|-------------|-------------|--------------------|-------------|------------|----------------|-----------------------|
| 1 | 0                     | abdominal_diameter | AAA             | CTRL            | 10          | 10          | 8.7055007          | 9.333229    | 8.83e-06   | 1.77e-05       | ****                  |
| 2 | 0                     | abdominal_diameter | AAA             | TAA             | 10          | 10          | 8.6463759          | 10.207335   | 5.16e-06   | 1.55e-05       | ****                  |
| 3 | 0                     | abdominal_diameter | CTRL            | TAA             | 10          | 10          | 0.5021738          | 13.599829   | 6.24e-01   | 6.24e-01       | ns                    |
| 4 | 1                     | abdominal_diameter | AAA             | CTRL            | 10          | 10          | 6.2883878          | 9.965409    | 9.18e-05   | 2.14e-04       | ***                   |
| 5 | 1                     | abdominal_diameter | AAA             | TAA             | 10          | 10          | 6.5975960          | 9.671458    | 7.13e-05   | 2.14e-04       | ***                   |
| 6 | 1                     | abdominal_diameter | CTRL            | TAA             | 10          | 10          | 0.8757762          | 17.433248   | 3.93e-01   | 3.93e-01       | ns                    |

6 rows

# Using the ggpubr package to plot significances

```
stats <- df_sub %>%  
  rstatix::t_test(abdominal_diameter ~  
diagnosis) %>%  
  add_xy_position()  
  
df_sub %>%  
  ggplot(aes(diagnosis, abdominal_diameter))+  
  geom_boxplot()+  
  geom_point()+  
  ggpubr::stat_pvalue_manual(stats)
```

Passing the data  
calling the *t\_test()* function  
formula: abdominal diameter is explained by diagnosis  
add x and y positions

Passing the data  
mapping the data  
boxplot  
add points  
add the p-values from the *stats df*

# Using the ggpubr package to plot significances

```
stats <- df_sub %>%  
  rstatix::t_test(abdominal_diameter ~  
diagnosis) %>%  
  add_xy_position()  
  
df_sub %>%  
  ggplot(aes(diagnosis, abdominal_diameter))+  
  geom_boxplot()+  
  geom_point()+  
  ggpubr::stat_pvalue_manual(stats)
```

Passing the data  
calling the `t_test()` function  
formula: abdominal diameter is explained by diagnosis  
add x and y positions

Passing the data  
mapping the data  
boxplot  
add points  
add the p-values from the *stats* df

ggpubr 0.6.1 Reference Changelog

## ggpubr: 'ggplot2' Based Publication Ready Plots

[ggplot2](#), by [Hadley Wickham](#), is an excellent and flexible package for elegant data visualization in R. However the default generated plots requires some formatting before we can send them for publication. Furthermore, to customize a ggplot, the syntax is opaque and this raises the level of difficulty for researchers with no advanced R programming skills.

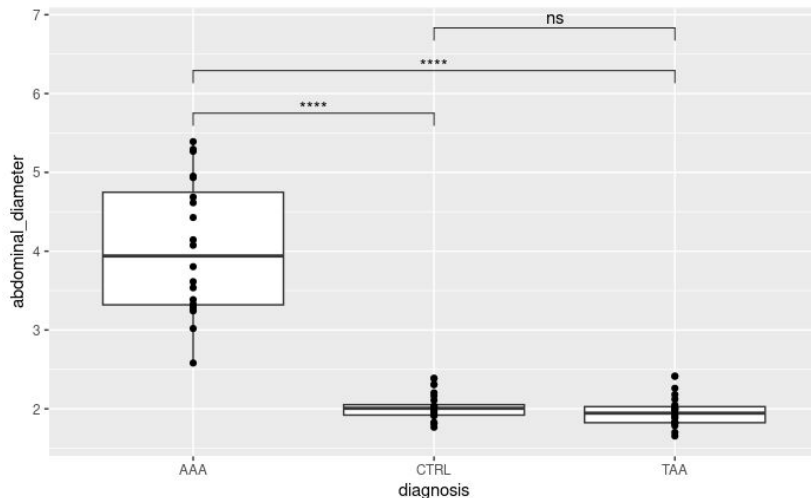
The 'ggpubr' package provides some easy-to-use functions for creating and customizing 'ggplot2' - based publication ready plots.

Find out more at <https://rpkgs.datanovia.com/ggpubr/>.



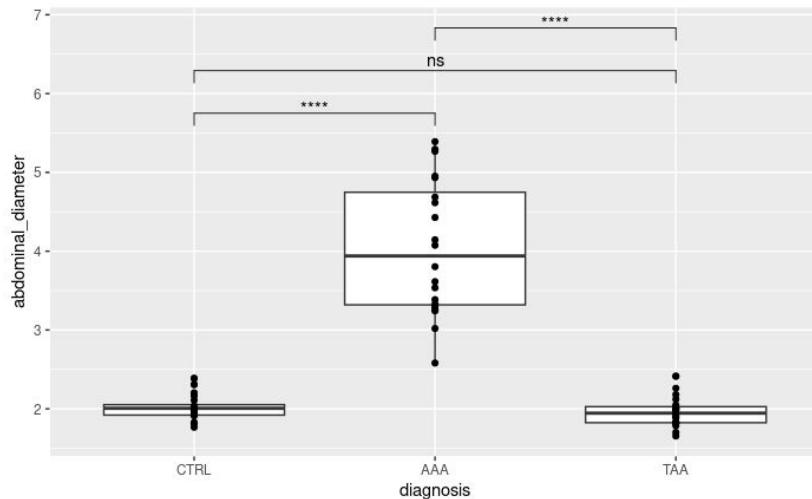
# Using the ggpubr package to plot significances

```
stats <- df_sub %>%  
  rstatix::t_test(abdominal_diameter ~  
diagnosis) %>%  
  add_xy_position()  
  
df_sub %>%  
  ggplot(aes(diagnosis, abdominal_diameter))+  
  geom_boxplot()+  
  geom_point()+  
  ggpubr::stat_pvalue_manual(stats)
```



# Using the ggpubr package to plot significances

```
stats <- df_sub %>%  
  mutate(diagnosis = fct_relevel(diagnosis,  
    "CTRL")) %>%  
  rstatix::t_test(abdominal_diameter ~  
    diagnosis) %>%  
  add_xy_position()  
  
df_sub %>%  
  mutate(diagnosis = fct_relevel(diagnosis,  
    "CTRL")) %>%  
  ggplot(aes(diagnosis, abdominal_diameter))+  
    geom_boxplot()+  
    geom_point()+  
    ggpubr::stat_pvalue_manual(stats)
```



# Summary

- project organisation using R projects and projects folder
- anatomy of a ggplot
- various geoms to visualise a variety of data
- ways to modify the resulting plots for visual appearance
- mean separation and adding significances to plots

## Medium

---

M1: Create a box plot of abdominal aortic diameter. Reorder the X axis to have the CTRL diagnosis be the reference level

M2: Recreate the scatter plot from above. Color the points by diagnosis.

M3: Adjust the color code to something of your liking.

M4: Use `geom_histogram()` to visualise the distribution of abdominal diameter. Facet using the diagnosis.

M5: Sample your data to 10 observations per diagnosis. Use `stat_summary()` to generate a bar chart of thoracic diameter by diagnosis. Add error bars and individual observations.

M6: Building on the previous task, improve the looks of the previous plot like you would for an important presentation. Adjust Axis labels, fix the order on the x axis, give the plot a title. Color the bars by diagnosis. Try to make it as pleasing as possible (in your eyes).

M7: Sample your data to 3 observations per diagnosis. Test the difference in thoracic diameter between the diagnoses. Is the difference significant? Repeat the sampling and test again. Increase the number of observations per diagnosis to 5 and 10. How do the p-values change?

## Hard

---

H1: What happens when you run the following code?

```
df <- read_tsv(here("data", "aneurysm_survey", "example_data.txt"))
df_plot <- df %>%
  group_by(diagnosis, hypertension) %>%
  slice_sample(n = 5) %>%
  ungroup() %>%
  mutate(hypertension = ifelse(hypertension, "present", "normal bp"))

df_plot %>%
  ggplot(aes(hypertension, thoracic_diameter, color = diagnosis))+
  geom_boxplot()
```

H2: Try adding points to the plot using `geom_point()`. What happens? Can you fix their position? Hint: look at the `position` argument of `geom_point()`.

H3: Using what we learned last session, build an expression plot of *IL6* using the airway data. Best use `geom_boxplot()` and `geom_point()`.

H4: Can you add the significance to the plot?